

中国科学技术大学

专业硕士学位论文



.NET 底层 JIT 编译器在 LoongArch 上的分析与完善、优化

作者姓名：

专业类别： 电子信息

专业领域： 计算机技术

校内导师：

实践导师：

完成时间： 二〇二五年四月五日

University of Science and Technology of China
A dissertation for master's degree



Analysis, improvement, and optimization of underlying JIT compiler of .NET on LoongArch

Author:

Specialty: Electronic Information

Major: Computer Technology

Supervisors:

Instructor:

Finished time: April 5th, 2025

中国科学技术大学学位论文原创性声明

本人声明所呈交的学位论文,是本人在导师指导下进行研究工作所取得的成果。除已特别加以标注和致谢的地方外,论文中不包含任何他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的贡献均已在论文中作了明确的说明。

作者签名: _____

签字日期: _____

中国科学技术大学学位论文授权使用声明

作为申请学位的条件之一,学位论文著作权拥有者授权中国科学技术大学拥有学位论文的部分使用权,即:学校有权按有关规定向国家有关部门或机构送交论文的复印件和电子版,允许论文被查阅和借阅,可以将学位论文编入《中国学位论文全文数据库》等有关数据库进行检索,可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。本人提交的电子文档的内容和纸质论文的内容相一致。

控阅的学位论文在解除后也遵守此规定。

☐ 公开 ☐ 控阅 (____ 年)

作者签名: _____

导师签名: _____

签字日期: _____

签字日期: _____

摘要

龙芯中科近年推出了我国自主可控的 CPU 指令系统架构——LoongArch(龙架构)，正在全面推进软件生态建设，应用广泛的 .NET 平台的适配是一个重点方向。目前已经完成基本移植，但适配程度相比其他成熟架构还有一定差距，还需进行深度优化、完善支持更多扩展功能特性。

本文聚焦于 LoongArch 架构平台上 .NET 的 CoreCLR 运行时中的底层 RyuJIT 编译器模块，力图在已有版本的基础上优化其编译表现，主要工作包括：

(1) JIT 编译生成指令的分析

从批量统计应用程序编译所得指令的数量入手，对比了 LoongArch 和 Arm 上结果数据的差异，发现 LoongArch 上确实存在较大劣势，相同程序编译生成的指令数量普遍更多。进而详细分析一些典型用例程序的指令序列，基于访存寻址方式、大块连续内存初始化、小尺寸数据类型在寄存器中的表示、转移指令的利用等场景或问题，提出了一些改进完善 RyuJIT 编译器的优化构想。

(2) 一个缓冲区溢出防护机制失效漏洞的发现、分析与修复

在部分特殊场景下，会出现函数返回地址被缓冲区溢出篡改而编译器内置的 StackGuard 防护机制未起效的现象。分析发现根源在于栈帧布局设计存在缺陷——帧指针 FP/返回地址 RA 寄存器组的排布位置不当。为修复该漏洞，完善了栈帧布局的设计，新增实现了一种例外情况布局作为应对。修复完成后测试验证，效果正常符合预期。此外还结合指令特性拓展延伸提出了进一步优化栈帧布局的方向。

(3) 硬件内部函数和向量 SIMD 操作硬件加速特性的支持

通过在标准库中添加 API 定义和改造编译器，新增支持了 231 个硬件内部函数，加速了 107 个向量 SIMD 操作，覆盖面与其他成熟架构达到了同等水平，初步支持了该特性。测试结果显示，实现支持后运行时 runtime 的功能正确性维持了与实现支持前相同的水平，而综合性能则提升到了之前的约 1.213 倍，其中单个测试项性能提升的最大幅度更是达到了约 50 倍。

由于暂无其他团队从事相关工作，以上工作内容在一定程度上是“开创性的”，其中指令分析部分的成果为未来继续改进完善 RyuJIT 编译器提供参考，具有一定的指导意义；漏洞修复和硬件加速特性支持的成果已落实于代码，进入 LoongArch 平台的 .NET 运行时产品内部，提升了其安全性和性能，已经或即将发布、投入实际应用。

关键词：.NET；即时编译器；龙架构

ABSTRACT

Loongson Technology has recently introduced LoongArch, an independently controllable CPU instruction set architecture of our country, and is currently comprehensively advancing the construction of its software ecosystem, in which the adaptation of the widely used .NET platform is a key direction. The basic porting has been completed so far, but there is still a certain gap in adaptation compared with other mature architectures. Further in-depth optimization and support for more extended functional features are needed.

This thesis focuses on the underlying RyuJIT compiler module in the CoreCLR runtime of .NET on LoongArch architecture platform, striving to optimize its compilation performance based on the existing version. The main work includes:

(1) Analysis of Instructions Generated by JIT

Starting with the batch statistics of the number of instructions generated from some application programs, a comparison was made between the results on LoongArch and Arm. It showed that LoongArch indeed has a significant disadvantage, with the number of instructions generated from the same program being generally larger. Then detailed analysis of the instruction sequences of some typical programs was conducted. Based on scenarios or issues such as addressing modes of memory access, initialization of large contiguous memory blocks, representation of small data types in registers, and the use of branch instructions, some optimization ideas for improving the RyuJIT compiler were proposed.

(2) Discovery, Analysis, and Fix of a Buffer Overflow Protection Mechanism Failure Vulnerability

In some special scenarios, it has been observed that the function return address can be tampered with by a buffer overflow, while the StackGuard protection mechanism built into the compiler fails to take effect. Analysis revealed that the root cause of the issue lies in the flawed design of the stack frame layout, specifically the improper arrangement of the frame pointer (FP) and return address (RA) register group. To fix this vulnerability, the design of the stack frame layout was improved by introducing an exception layout to handle such cases. Test after the fix confirmed that the solution worked as expected. Additionally, based on the characteristics of the instructions, further directions for optimizing the stack frame layout were proposed.

(3) Support for Hardware Intrinsics and Vector SIMD Operations as Hardware Acceleration Features

By adding API definitions in the standard library and modifying the compiler, 231 hardware intrinsics were newly supported, and 107 vector SIMD operations were accelerated. The coverage has reached a level comparable to other mature architectures, meaning that the features have been supported initially. Tests showed that after implementing the support, the functional correctness of the runtime remains at the same level as before, while the overall performance has been improved by approximately 1.213 times, and the maximum performance improvement for a single test item has even reached about 50 times.

Since no other team is engaged in relevant work, the above work is "groundbreaking" to a certain extent. The achievements in the instruction analysis section of this thesis provide a reference for the continued improvement and refinement of the RyuJIT compiler in the future and have certain guiding significance. The results of the vulnerability fix and support for hardware acceleration features have been implemented in code and integrated into the .NET runtime product for LoongArch platform, enhancing its security and performance, and have either been released or are about to be released for practical application.

KEY WORDS: .NET; JIT; LoongArch

目 录

第 1 章 绪论.....	1
1.1 LoongArch（龙架构）简介.....	1
1.1.1 LoongArch 架构的发展.....	1
1.1.2 LoongArch 指令集的技术特性.....	2
1.2 .NET 底层 JIT 编译器简介.....	3
1.2.1 .NET 平台.....	3
1.2.2 底层 JIT 编译器.....	4
1.3 本文工作内容.....	6
1.4 本文组织结构.....	7
第 2 章 JIT 编译生成指令分析.....	8
2.1 指令数量统计.....	8
2.1.1 统计方法.....	8
2.1.2 统计结果.....	9
2.2 指令序列优化分析.....	11
2.2.1 向量操作和专用硬件加速指令.....	12
2.2.2 访存寻址方式.....	12
2.2.3 大块连续内存初始化.....	14
2.2.4 小尺寸数据类型在寄存器中的表示.....	18
2.2.5 其他.....	19
2.3 本章小结.....	19
第 3 章 缓冲区溢出防护机制失效漏洞分析与修复.....	21
3.1 背景介绍.....	21
3.1.1 缓冲区溢出.....	21
3.1.2 StackGuard 防护技术.....	23
3.2 漏洞表现与分析.....	24
3.2.1 漏洞的表现.....	24
3.2.2 漏洞的根源分析.....	26
3.3 漏洞修复.....	29
3.3.1 调整栈帧布局.....	30
3.3.2 调整函数序言/尾声指令生成逻辑.....	31

3.4 修复结果与优化拓展.....	33
3.4.1 修复结果.....	33
3.4.2 进一步优化的思路拓展.....	34
3.5 本章小结.....	35
第 4 章 硬件加速特性支持.....	36
4.1 硬件内部函数与 SIMD 简介.....	36
4.2 .NET 中的相关设计.....	37
4.3 为 LoongArch 添加特性支持的实现.....	40
4.3.1 库函数接口.....	40
4.3.2 编译基础设施.....	41
4.3.3 编译过程.....	41
4.4 添加特性支持的结果.....	43
4.4.1 功能测试.....	43
4.4.2 性能测试.....	44
4.4.3 后续工作方向.....	47
4.5 本章小结.....	47
第 5 章 总结与展望.....	49
5.1 本文工作总结.....	49
5.2 未来展望.....	50
参 考 文 献.....	51
致 谢.....	54
在读期间发表的学术论文与取得的其他研究成果.....	55

第 1 章 绪论

本文主题是 .NET 底层 JIT 编译器在 LoongArch 上的分析与完善、优化，由于目前从事 .NET 在 LoongArch 上适配工作的仅有龙芯自有团队，而其之前也未专门开展相关系统性分析优化工作，因此本课题的研究现状可视为“无”或“空白”，而对于后文具体工作内容相关技术的研究现状或背景，将于相应章节中分别介绍，本章绪论仅介绍 LoongArch 和 .NET 底层 JIT 编译器这两个研究对象以及本文工作开展的整体背景与内容。

1.1 LoongArch（龙架构）简介

1.1.1 LoongArch 架构的发展

龙芯通用处理器是国内自主 CPU 的代表产品，其研发始于中国科学院计算技术研究所，早期产品采用了国外授权的 MIPS 指令集架构，并在后期添加了自定义扩展指令，以满足特定应用需求。然而，随着行业的不断进步和市场需求的日益变化，MIPS 指令集在性能扩展性、生态发展潜力等方面逐渐显现出局限性。例如，其指令集复杂度较高，部分指令在现代处理器架构下执行效率较低，且在与其它主流架构（如 x86、ARM）的兼容性方面存在不足。

在综合考虑了技术进展现状、MIPS 指令集的缺陷以及时代背景等因素后，为市场化运作龙芯成立的龙芯中科公司^[1]做出了重大决策，果断舍弃了 MIPS 指令集，转而基于长达二十年的 CPU 研制和生态建设技术积累，于 2020 年推出了自主设计的指令集架构——LoongArch（龙架构）。LoongArch 的目标是实现自主性、先进性、开放性和兼容性的统一，旨在构建一个能够与当前占据市场的 Wintel（Windows + Intel）和 AA（Android + ARM）体系相抗衡的自主可控的计算生态第三极，为国家信息产业安全提供坚实的底层技术支撑。

LoongArch 指令集架构的推出，标志着龙芯处理器在技术路线上的重大转变。新架构淘汰了传统指令系统中一些不适应当前软硬件设计技术发展趋势的陈旧内容，吸纳借鉴了近年来指令系统设计领域的诸多先进技术发展成果。与原有指令系统相比，不仅在硬件方面更易于高性能低功耗设计，而且在软件方面更易于编译优化和操作系统、虚拟机的开发。此外，LoongArch 在设计时充分考虑了兼容生态需求，融合了 x86、Arm、MIPS 等国际主流指令系统的主要功能特性，同时基于在二进制翻译领域长期的技术积累创新，能够实现对基于其他指令系统架

构的应用程序的高效二进制翻译^[2]。

在生态建设方面,龙芯中科采取了一系列积极措施。一方面,通过开源策略,免费开放 LoongArch 架构,吸引了众多开发者和企业的参与。另一方面,利用二进制翻译技术,实现了对 MIPS、x86、ARM 等主流指令集的高效兼容,这使得大量现有的软件能够较容易地实现在 LoongArch 架构上运行,极大地降低了软件迁移成本。目前,龙芯中科正在全力推动 LoongArch 架构的软件生态发展,包括操作系统适配、编译器优化、应用软件移植等多方面工作,并已取得显著成效,得到了国际开源软件界的广泛认可与支持,如 Linux、Binutils、GDB、.NET、GCC、LLVM、Go、Chromium/V8、Mozilla/SpiderMonkey、FFmpeg、libyuv、libvpx、OpenH264、SRS、UEFI、龙蜥、openEuler 等。

随着 LoongArch 架构的不断完善和推广,基于该架构的 CPU 产品也在不断更新换代。从龙芯 3A5000 一代全面转向 LoongArch 架构开始,龙芯处理器在性能、功耗、安全性等方面均取得了显著提升,获得了市场认可,LoongArch 生态系统正在逐步走向繁荣。

1.1.2 LoongArch 指令集的技术特性

LoongArch 采用了精简指令集计算机(Reduced Instruction Set Computer, RISC)的设计风格,指令长度固定且编码格式规整,仅有 load/store 类访存指令可以访问内存。指令集共计近 2000 条指令,采用模块化组织形式,包括基础架构部分(Loongson Base)和向量扩展、虚拟化扩展(Loongson Virtualization, LVZ)、二进制翻译扩展(Loongson Binary Translation, LBT)三个扩展部分,分别用于加速计算密集型应用、操作系统虚拟化、跨指令系统二进制翻译,其中向量扩展部分又细分为 128 位向量扩展(Loongson SIMD eXtension, LSX)和 256 位向量扩展(Loongson A dvanced SIMD eXtension, LASX)。硬件设计时除基础部分必需外,各扩展部分可按需灵活选择实现(实现 LASX 时必须同时实现 LSX)。

LoongArch 指令集^[3]中的所有指令长度均为 32 位,且要求指令地址 4 字节边界对齐。在指令编码方面,大多数指令为三操作数,所有寄存器操作数域都从第 0 位开始从低到高依次摆放,操作码都从第 31 位开始从高到低依次摆放。如果指令中包含立即数操作数,那么立即数域位于寄存器域和操作码域之间,根据不同指令类型有不同的长度。具体来说,包含 9 种典型的指令编码格式,即 3 种不含立即数的编码格式 2R、3R、4R,以及 6 种含立即数的编码格式 2RI8、2RI12、2RI14、2RI16、1RI21、I26。

LoongArch 下的寄存器包括通用寄存器(GPR)、程序计数器(PC)、浮点寄存器(FPR)、条件标志寄存器(CFR)、控制状态寄存器(CSR)和向量寄

寄存器（VR）等。其中通用寄存器有 32 个，宽度为 32 或 64 比特，浮点寄存器有 32 个，宽度通常为 64 比特，内核使用的寄存器约定定义在 LoongArch ELF psABI 规范^[4]中。向量寄存器有 32 个，在 LSX 中宽度为 128 比特，其低半部分复用浮点寄存器，在 LASX 中宽度为 256 比特，其低半部分复用 LSX 中的向量寄存器。

LoongArch 按照地址空间大小可分为 32 位和 64 位两个版本，分别称为 LoongArch32 和 LoongArch64，LoongArch64 应用级向下二进制兼容 LoongArch32。本文工作内容均基于 64 位版本 LoongArch64，为了行文简洁，后文中可能会省略 64 简写为 LoongArch，但所指代对象不变。

1.2 .NET 底层 JIT 编译器简介

1.2.1 .NET 平台

.NET(dotnet)^[5] 是最初由微软（Microsoft）公司推出、现在由其和开源社区共同维护的免费、开源、跨平台的框架，可用于构建多种类型的应用或服务，具有庞大的支持生态系统和强大的工具，在行业中应用广泛并仍然在蓬勃发展中。.NET 平台灵活性高、功能强大、性能优越、安全可靠，部分设计亮点包括：

- 技术生态中包括运行时、库、语言和各种辅助工具，提供全栈的生产力；
- 支持 C#、F#、Visual Basic 三种编程语言编写程序，其中 C# 最为常用；
- 由于使用了垃圾回收（Garbage Collection，GC）机制和严格的语言编译器，.NET 的类型安全、内存安全（也允许高级开发者使用“不安全代码”手动进行额外的优化）；
- 预先（Ahead Of Time，AOT）编译为静态代码和即时编译（Just-in-Time，JIT）动态代码两种机制均支持，可适应不同的应用场景；
- 既支持便捷的跨平台通用开发，也支持利用硬件内部函数等机制针对特定平台开展专门的深度优化。

.NET 诞生于 2000 年，早期版本称为 .NET Framework，仅支持 Windows 操作系统，但微软公司推动将相关的公共语言基础结构（Common Language Infrastructure，CLI）通过 ECMA^[6]和 ISO^[7]进行了标准化。其他公司以此为基础创建了对应的跨平台实现——开源项目 Mono^[8]，并获得成功。2016 年微软推出了官方的跨平台版本 .NET Core，并开源了代码。2020 年微软将 .NET Framework、.NET Core 和之前通过收购接管的 Mono 整合为了跨平台的单一 .NET，并继续维持开源。在长期的发展过程中，.NET 持续增强改进、扩展新功能，生态也不断壮大，成为软件开发领域的一个重要平台。

从技术层面而言，.NET 平台包括以下组件：

- 运行时（runtime），用于运行应用程序。
- 类库，以模块形式向开发者提供实用功能组件。
- （上层）编译器，将使用 C# 等语言编写的源代码编译为运行时可执行的中间语言代码 CIL（Common Intermediate Language，通用中间语言）。
- SDK 和其他工具，支持开发者构建生成和管理应用程序。
- 应用开发框架，例如 ASP.NET Core 和 Windows 窗体。

应用程序代码的核心流转过程为：上层 C# 等语言的源代码（调用类库）经（上层）编译器编译为 CIL 字节码，字节码经运行时（runtime）中的底层编译器再次编译生成最终的机器指令，再由运行时中的虚拟机（执行引擎）调度给操作系统和硬件执行。

.NET 的运行时（runtime）^[9]是公共语言基础结构 CLI 的核心内容落地而成的产品（在 CLI 中称为 CLR，Common Language Runtime），由于前述的历史原因，主要有三个变体——.NET Framework、Mono 和 .NET Core 各自内部的运行时，整合为统一的 .NET 平台之后三者仍保持存在，但实际上后续的发展将主要以原来的 .NET Core 为基础。本文相关工作基于的便是整合之后的 .NET，其中源于 .NET Core 的运行时组件称为 CoreCLR。

作为 CLR 标准的一个具体实现，CoreCLR 运行时整体上的功能特性已由相关规范确定。CLR 是一个完整的、高级的（广义）虚拟机，旨在支持多种编程语言及其之间的互操作。CLR 管理应用程序（字节码级别）从构造和绑定到部署和执行的整个生命周期，定义了一个带有垃圾回收（GC）机制的虚拟机、一种程序声明的元数据的表示方式、一种 CLR 可执行程序文件的格式、以及运行期间程序相互引用等操作的实现机制等。具体而言，CLR 的功能包括垃圾回收（GC）、内存安全、类型安全、字节码编译、对编程语言的高级支持（面向对象、值类型、异常处理、泛型、反射等）、程序隔离、调试、性能分析、支持托管代码与非托管代码的互操作、支持多线程机制等等。

1.2.2 底层 JIT 编译器

CoreCLR 运行时内部通过即时（Just-In-Time，JIT）编译器——RyuJIT 将 CIL 字节码编译为最终的机器指令。RyuJIT 通过接口与虚拟机（执行引擎）交互，提供编译服务。

即时编译 JIT，是一种在程序到运行时才对源代码进行编译的机制，广泛应用于 .NET 和 Java 等现代的编程语言运行时环境中。与传统的预先（Ahead-Of-Time，AOT）编译方式不同，JIT 编译器在程序执行过程中动态地将中间字节码

转换为机器码，从而能够在运行时根据程序的实际执行情况开展优化，提升程序的性能，这是其核心优势所在。具体而言，由于程序的运行时行为可能与编译时的假设存在差异，JIT 编译器能够通过收集运行时状态信息（如热点代码、分支预测等）来动态调整代码的编译策略。例如，对于频繁执行的代码片段（热点代码），JIT 编译器可以分层编译，在检测到其是热点后投入更多资源、采用更复杂的编译算法对其进行深度优化。此外，JIT 编译器还可以利用运行期间检测到的硬件信息（如 CPU 特性）来生成针对特定平台优化、加速的机器指令，进一步提升程序性能。

当然，JIT 编译也存在一些局限性。JIT 编译过程由于是在程序运行时进行的，其自身也会占用一定的运行时资源，包括 CPU 时间和内存空间等，这可能导致程序在启动初期或首次执行某些代码时出现短暂的性能延迟。此外，JIT 编译器的优化策略依赖于运行时收集的信息，对于一些复杂的程序逻辑或动态变化较大的代码，优化效果可能不如 AOT 编译器那样稳定。尽管如此，JIT 编译技术仍然具有重要的价值和地位，它为跨平台编程语言提供了高效的运行时支持，使得开发者能够在不同的硬件平台上获得较为一致的体验。同时，随着技术的不断发展，通过采用多级分层编译策略、延迟编译和代码缓存等措施，JIT 编译器的优化能力和运行时性能也在持续提升，运行时资源占用等问题也得到了有效缓解。

RyuJIT 编译器是从 .NET Framework 上用于 x86 的 JIT —— jit32 演变而来的，并被移植以支持曾经 .NET Core 支持的所有其他架构和目标平台，如今仍在不断发展完善、提升性能、拓展功能。

RyuJIT 的整体编译流程如图 1.1 所示，可分为前端、中端、后端三部分 15 个阶段。前端负责解析 IL、创建和规范化 IR（intermediate representation，中间表示）与关联的数据结构，中端开展一些典型优化，后端进行较低级别的处理并

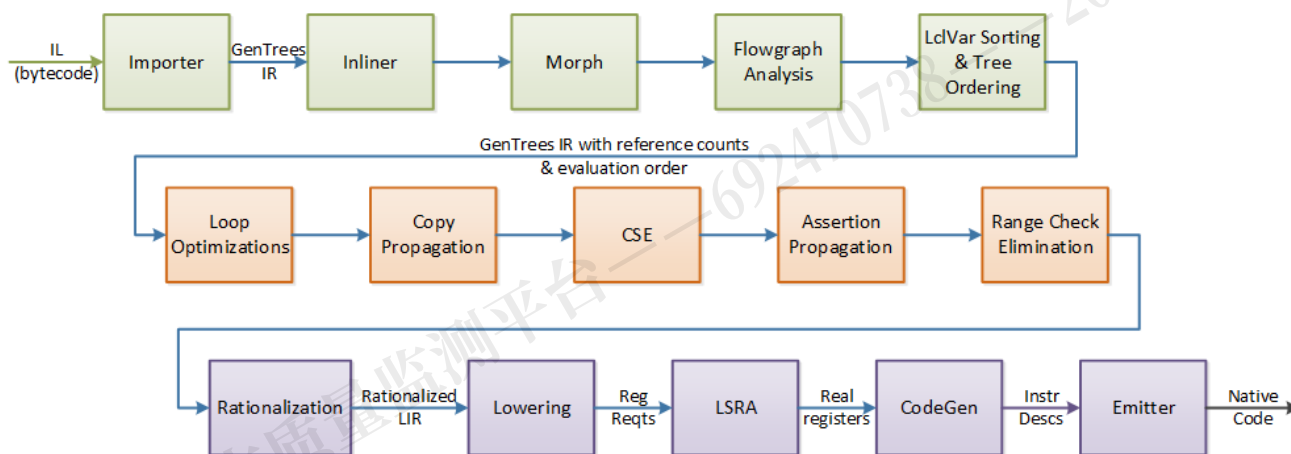


图 1.1 RyuJIT 编译器的编译流程

生成和发出机器指令，其中后端是移植适配新指令集架构时重点做的部分。

前端的 **Importer** 导入阶段负责初始化局部变量表，解析 **IL** 构造基础块（**BasicBlocks**）、创建高级 **IR**。**Inliner** 内联阶段对函数调用适当进行内联处理。**Morph** 变形节点对 **IR** 进行一些变换以便后续优化。**Flowgraph Analysis** 流程图分析阶段分析各基础块的流程。**LclVar Sorting & Tree Ordering** 本地变量排序和评价顺序定义阶段统计本地变量的引用计数并排序，指定 **IR** 中节点的评价顺序。

中端的各阶段基于生命周期分析、变量版本标记和值编号机制开展循环优化、复制传播、公共子表达式消除、断言传播、边界检查消除等一系列优化。

后端的 **Rationalization** 合理化阶段将高级 **IR** 变形生成低级 **IR**。**Lowering** 低级化阶段转换 **IR** 以暴露控制流和寄存器需求。**LSRA** 线性扫描寄存器分配阶段为语法树节点和变量分配寄存器。**CodeGen** 目标代码生成阶段确定栈帧布局，根据低级 **IR** 生成汇编指令和函数的序言与尾声，以及与调试、垃圾回收、异常处理等相关的信息。**Emitter** 指令发射阶段根据汇编指令生成机器码并发射。

1.3 本文工作内容

如前所述，目前 **LoongArch** 架构正在全面推进软件生态建设，而 **.NET** 平台又在软件领域占据重要地位，因此，实现对 **.NET** 的良好兼容、高效支持，从而承接 **.NET** 应用生态体系，是非常有意义、有必要的。

龙芯中科团队从之前采用 **MIPS** 架构的时期便已开始对 **.NET**（前身 **.NET Core**）的相关移植工作^[10]，具有丰富的技术和经验积累，在推出自主的 **LoongArch** 架构后又快速实现了 **.NET** 在 **LoongArch** 上的基本适配，并公开发布了相应 **SDK** 产品^[11]，进入了实际市场应用，相关工作也得到了开源社区的官方认可、接纳与支持。新的工作重点是在与开源社区的产品主体保持同步更新的同时基于 **LoongArch** 架构自身特点进行深度优化、完善支持扩展特性等，提高性能表现、稳定性和安全性等。

本文工作即以实现基本适配的 **.NET 6.0** 版本为基础，对运行时 **runtime** 开展分析与优化，并侧重于底层的 **RyuJIT** 编译器。**6.0** 版本为相关工作开展时 **LoongArch** 架构上已完成基本移植适配的最新 **LTS**（**Long-Term Support**，长期支持）版本，而且由于 **JIT** 编译器作为底层核心部件，不会频繁大改动，本文所做工作稍作调整也可应用于后续更新的版本或为其提供指导参考。

主要工作内容包括：

- (1) **JIT** 编译生成指令的分析：静态指令数量统计、指令序列优化探究。
- (2) 一个缓冲区溢出防护机制失效的安全漏洞的发现、分析与修复。

- (3) 添加支持硬件加速特性——硬件内部函数（`hardware intrinsic`）与向量 SIMD 操作。

以上构成本文“分析与完善、优化”的三方面工作表面上相互较为独立，但内在具有一定的递进关系。指令分析部分中有一些场景的优化方案正对应着后两项工作：正是对于访存寻址方式的分析探究促成了安全漏洞的发现；向量操作和专用硬件加速指令相关用例程序在 LoongArch 上的指令数量劣势较大正是添加支持相关硬件加速特性的动机。

1.4 本文组织结构

第 1 章为绪论，介绍研究对象 .NET 底层 JIT 编译器和 LoongArch 指令集架构、本文的工作内容。

第 2 章介绍工作内容一 —— JIT 编译器生成指令的分析与优化探究。

第 3 章介绍工作内容二 —— 对 JIT 编译器中一个安全漏洞的分析与修复，包括漏洞涉及技术的背景信息、暴露表现、产生原因分析、修复措施与结果、以及优化方向拓展等。

第 4 章介绍工作内容三 —— 添加硬件内部函数与向量 SIMD 操作硬件加速特性支持，包括技术背景、相关机制、实现过程、支持的结果和测试表现。

第 5 章为对以上所做工作的总结及未来展望。

第 2 章 JIT 编译生成指令分析

编译器的本质用途是将高级语言翻译成机器能识别的低级指令，因此评价编译器的一个重要指标便是其生成指令的质量，即在实现同一功能（翻译同一段高级语言）的情况下是否采用了最优的指令选择与组合方案，通过对此进行分析，可以发掘出改进完善编译器的方向。

LoongArch 架构才适配 .NET 不久，包括底层 RyuJIT 编译器在内的各组件也尚未系统性地开展优化。本章阐述对 RyuJIT 编译生成指令的分析工作，其中 2.1 节从宏观层面大批量统计应用程序经 JIT 编译生成静态指令的数量，从而筛选出优化潜力较大的典型场景，2.2 节分析具体指令序列的优化可能性和方向，2.3 节对本章内容作简要总结。

2.1 指令数量统计

编译所得静态指令的数量直观体现编译器的综合编译效果，一定程度上也反映其性能表现。通过对比不同架构上 RyuJIT 编译器编译若干相同程序所得指令的数量差异，有助于快速筛选出一些包含潜在优化点的程序，进而基于其展开分析，指导对编译器的优化。

相比于动态指令，静态指令的数量确实对于程序性能的反映更加不准确，有一定的局限性，例如有些指令最终并不会进入处理器实际执行、循环结构会将大量动态指令“压缩”为少量静态指令等，但这只是准确程度的高低问题，对于相同的源码生成的多份静态指令，其数量的差异仍能定性反映编译器的整体性能差别，而且数量不同导致的指令占用存储空间大小的不同，也可以看作广义的性能。此外，而且对于那些并非少数的、线性的普通逻辑，静态指令和动态指令也是一致的。因此，分析静态指令是有价值的。

另外，在技术层面上，JIT 编译器编译的是由虚拟机托管的代码，统计时要将其与原生（native）代码区分开需要一些辅助工具支持，但目前由于生态适配不足，暂时也无法进行准确的动态指令统计。不过，在后续分析具体指令序列时会综合考虑动态指令的情况。

2.1.1 统计方法

runtime 源码仓库中内嵌有一个功能性验证测试用例集，广泛覆盖了

CoreCLR 的方方面面，尤其是测试 JIT 编译器的部分，覆盖了各种编译场景，正适合作为生成指令分析的对象。基于该测试集开展指令数量统计工作：

- 1) 利用 RyuJIT 编译器的反汇编功能，在用例程序运行期间打印输出编译所生成的汇编指令的基本统计数据——单个方法（method）对应的指令数量。
- 2) 编写脚本，通过字符正则匹配方式提取出打印输出内容中的各方法（method）指令数量的数据值，汇总加和算得单个用例程序的指令总数。
- 3) 对整个测试集开展批量统计，获得全体用例程序的指令总数数据并记录保存。
- 4) 在其他平台架构上采用同样方法、相同脚本程序完成统计，对比分析所得数据。

由于 RyuJIT 编译器具有分层编译等机制，对于同一方法（method）启用优化选项前后生成的指令有所不同，每次开展统计所得到的数据有一定的波动变化，实际统计时采用了多次取平均值的策略。另外，部分用例程序源码中包含打印语句，其自身输出的内容会与编译器反汇编功能的输出相混淆，妨碍数据的提取，好在测试用例数量非常多（有超过一万个），直接剔除了受影响的用例数据。

对比分析选用了 Arm64 架构平台，而未选择 x64（x86_64/AMD64），是因为 Arm 与 LoongArch 同属 RISC 类型指令集，具有更大的相似性，而 x64 为与之相对的 CISC（Complex Instruction Set Computer）类型，本质差异过大，可比性过小。当然，即使同为 RISC 类型，指令集具体设计的不同也导致无法精确地只对比由编译器编译策略造成的指令数量差异，但指令数量统计工作本身其实只是为了发掘潜在优化点，只要能显示出趋势、整体分布情况以及有助于筛选出差异明显的典型用例程序即可。

2.1.2 统计结果

分别汇总整理 LoongArch64 和 Arm64 上的统计结果数据，并以 Arm64 上的数据为基准进行对比，数据有效且可比的 5191 个用例程序中：

（1）LoongArch64 上生成指令数量少于 Arm64 的用例程序共 5 个，用例程序指令总数的差值所占比例的最小值约为 1.65%，最大值约 46.35%，具体数据如表 2.1 所示。

表 2.1 生成指令在 LoongArch 上比在 Arm 上少的用例程序数据

用例程序	LA 上 指令数	Arm 上 指令数	指令数 差值 ^①	差值占 Arm 比 例
baseservices/exceptions/simple/ParallelCrashWorkerThreads/ParallelCrashWorkerThreads.sh	7446	7571	-125	-1.65%
baseservices/exceptions/simple/ParallelCrashMainThread/ParallelCrashMainThread.sh	7446	7575	-129	-1.70%
JIT/SIMD/VectorArray_r/VectorArray_r.sh	38370	43209	-4839	-11.19%
JIT/SIMD/VectorArray_ro/VectorArray_ro.sh	26863	38931	-12068	-30.99%
baseservices/TieredCompilation/BasicTestWithMcj/BasicTestWithMcj.sh	66193	123384	-57191	-46.35%

注：①指令数差值为 LoongArch 上指令数减去 Arm 上指令数。

(2) LoongArch64 上生成指令数量多于 Arm64 的用例程序共 5186 个。用例程序指令总数的差值所占比例的最小值约为 0.36%，最大值达到约 823.71%，用例程序数量在各差值所占比例区间下的分布情况如图 2.1 所示，可见数量差异百分比集中于 40% ~ 60% 区间，比例差别最大的前 20 个用例程序的数据如表 2.2 所示。

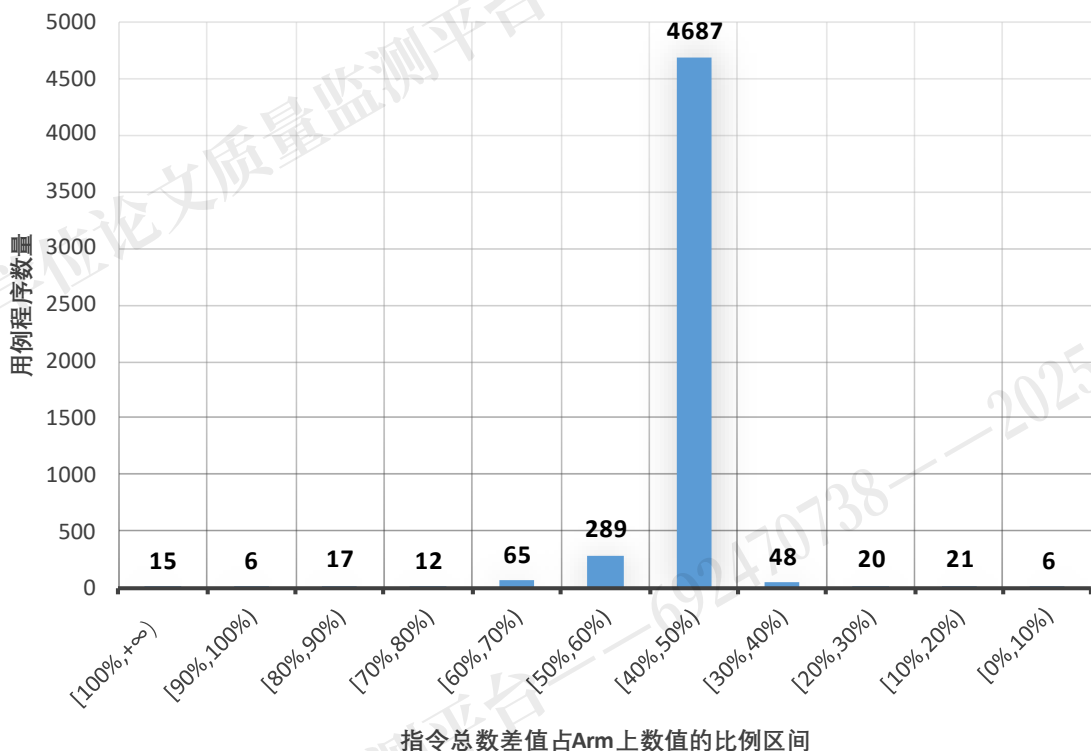


图 2.1 测试集程序在 LoongArch 与 Arm 架构上编译所得指令数量差异情况分布

表 2.2 生成指令在 LoongArch 上比在 Arm 上多的部分用例程序数据

用例程序	LA 上 指令数	Arm 上指令 数	指令数 差值 ^①	差值占 Arm 比例
Regressions/coreclr/GitHub_22348/Test22348/Test22348.sh	59404	6431	52973	823.71%
JIT/Regression/CLR-x86-JIT/V1-M12-Beta2/b57518/b57518/b57518.sh	41994	6116	35878	586.62%
JIT/SIMD/VectorUnused_ro/VectorUnused_ro.sh	26118	6627	19491	294.11%
JIT/SIMD/VectorUnused_r/VectorUnused_r.sh	26565	7252	19313	266.31%
JIT/HardwareIntrinsics/X86/Regression/GitHub_17073/GitHub_17073/GitHub_17073.sh	101438	33495	67943	202.84%
JIT/SIMD/VectorExp_ro/VectorExp_ro.sh	16051	6468	9583	148.16%
JIT/SIMD/VectorExp_r/VectorExp_r.sh	17099	6950	10149	146.02%
JIT/SIMD/AbsGeneric_ro/AbsGeneric_ro.sh	14830	6225	8605	138.23%
JIT/SIMD/AbsGeneric_r/AbsGeneric_r.sh	14918	6331	8587	135.63%
JIT/SIMD/SqrtGeneric_ro/SqrtGeneric_ro.sh	15344	6756	8588	127.11%
JIT/SIMD/SqrtGeneric_r/SqrtGeneric_r.sh	15461	6866	8595	125.18%
JIT/SIMD/VectorSub_r/VectorSub_r.sh	25156	11860	13296	112.10%
JIT/SIMD/CreateGeneric_r/CreateGeneric_r.sh	13394	6362	7032	110.53%
JIT/Regression/JitBlue/GitHub_11816/GitHub_11816/GitHub_11816.sh	13054	6352	6702	105.51%
JIT/Regression/CLR-x86-JIT/V1.2-Beta1/b191926/b191926/b191926.sh	41251	20425	20826	101.96%
JIT/HardwareIntrinsics/X86/Avx2/GatherMaskVector128_r/GatherMaskVector128_r.sh	35227	17700	17527	99.02%
JIT/Directed/StructABI/structreturn/structreturn.sh	64653	32810	31843	97.05%
JIT/Regression/CLR-x86-JIT/V1-M12-Beta2/b71005/b71005/b71005.sh	65096	33248	31848	95.78%
JIT/Methodical/largeframes/skip4/skippage4/skippage4.sh	52242	26784	25458	95.04%
JIT/SIMD/VectorSqrt_ro/VectorSqrt_ro.sh	12048	6277	5771	91.93%

注：①指令数差值为 LoongArch 上指令数减去 Arm 上指令数。

整体上看，测试用例集中的程序在 LoongArch64 架构上编译所得静态指令数普遍要比在 Arm64 架构上更多，仅极少量用例程序 LoongArch 有优势。因此，LoongArch 上的 RyuJIT 编译器很可能实现得不够充分，有很大的优化空间，亟需深入分析、改进完善。

2.2 指令序列优化分析

基于前述指令数量统计情况，优先从 LoongArch 上指令数量劣势较大的那

些典型用例程序入手，详细分析其对应生成的指令序列，结合架构特点从指令的选择与组合角度发掘出一些潜在优化点，并初步设计改进方案，为后续不断完善 RyuJIT 编译器提供指导。

说明：限于工程任务优先级、辅助研发工具缺少适配等因素，以下分析中所提出的优化方案设想大多暂未落地实现，仅是为未来工作提供参考。

2.2.1 向量操作和专用硬件加速指令

从表 2.2 中的部分用例程序的路径和命名中即可发现，相当一大部分 LoongArch 劣势场景与向量操作（SIMD）和硬件内部函数（HardwareIntrinsics）有关，而 LoongArch 上尚未适配支持该特性，后文第 4 章介绍相关原理和开展完成的工作。

2.2.2 访存寻址方式

在用例程序生成的指令中，有一类访存读写大尺寸函数栈帧上数据的场景，如果栈帧尺寸较大（超过 2048 字节），在访问过大的部分空间时当前 LoongArch 上的指令序列较为繁琐，需要比 Arm 多使用 2 条指令（lu12i、add）用于辅助构造访存地址，一个简单的例子示意如表 2.3 所示。

表 2.3 在大尺寸栈帧上存取数据时 LoongArch 和 Arm 所用指令的差异

LoongArch			Arm
lu12i.w	t1, 0x1	；寄存器 t1 中写入值 0x1000(=4096)	str x0, [fp, 3520]
add.d	t1, t1, fp	；寄存器 t1 中值加上 fp 的值，等于 fp+4096	
st.d	a0, t1, -576	；最终地址为 t1 中值减去 576，等于 fp+3520	

原因在于，访存位置的地址与作为偏移基准的帧指针 fp 的差值过大，超出了 st 指令自身所支持的范围，即对于上述例子，在 LoongArch 上无法构造出指令“st.d a0, fp, 3520”。

访问内存读写数据是程序中的高频操作，LoongArch 指令集中的 6 类访存指令如表 2.4 所示。

表 2.4 LoongArch 指令集中的访存指令

分类	指令	指令格式	
普通访存	ld	LD.{B[U]/H[U]/W[U]/D}	rd, rj, si12
	st	ST.{B/H/W/D}	rd, rj, si12
	ldx	LDX.{B[U]/H[U]/W[U]/D}	rd, rj, rk
	stx	STX.{B/H/W/D}	rd, rj, rk
	ldptr、stptr	{LD/ST}PTR.{W/D}	rd, rj, si14
	preld	PRELD	hint, rj, si12
	preldx	PRELDX	hint, rj, rk
边界检查 访存	ldgt、ldle、stgt、stle	{LD/ST}{GT/LE}.{B/H/W/D}	rd, rj, rk
原子访存	amswap、amadd、 amcas	AM{SWAP/ADD/CAS}[DB].{B/H/ W/D}	rd, rk, rj
	amand、amor、amxor	AM{AND/OR/XOR}.{W/D}	rd, rk, rj
	amax、amin	AM{MAX/MIN}[DB].{W[U]/D[U]}	rd, rk, rj
	ll、sc	{LL/SC}.{W/D}	rd, rj, si14
	sc.q	SC.Q	rd, rk, rj
	ll.acq	LL.ACQ.{W/D}	rd, rj
	sc.rel	SC.REL.{W/D}	rd, rj
浮点普通 访存	fld、fst	F{LD/ST}.{S/D}	fd, rj, si12
	fldx、fstx	F{LD/ST}X.{S/D}	fd, rj, rk
浮点边界 检查访存	fldgt、fldle、 fstgt、fstle	F{LD/ST}{GT/LE}.{S/D}	fd, rj, rk
向量访存	vld、vst	V{LD/ST}	vd, rj, si12
	xvld、xvst	XV{LD/ST}	xd, rj, si12
	vldx、vstx	V{LD/ST}X	vd, rj, rk
	xvldx、xvstx	XV{LD/ST}X	xd, rj, rk
	vldrepl.b	vldrepl.b	vd, rj, si12
	vldrepl.h	vldrepl.h	vd, rj, si11
	vldrepl.w	vldrepl.w	vd, rj, si10
	vldrepl.d	vldrepl.d	vd, rj, si9
	xldrepl.b	xldrepl.b	xd, rj, si12
	xldrepl.h	xldrepl.h	xd, rj, si11
	xldrepl.w	xldrepl.w	xd, rj, si10
	xldrepl.d	xldrepl.d	xd, rj, si9
	vstelm.h	vstelm.h	vd, rj, si8, idx
	xvstelm.h	xvstelm.h	xd, rj, si8, idx

注：指令格式中，“.XX”形式的后缀表示指令的操作对象类型，对于整数类型指令，.B、.H、.W、.D、.BU、.HU、.WU、.DU 分别表示有符号字节、有符号半字、有符号字、有符号双字、无符号字节、无符号半字、无符号字、无符号双字；对于浮点数据类型指令，.H、.S、.D、.W、.L、.WU、.LU 分别表示半精度浮点数、单精度浮点数、双精度浮点数、有符号字、有符号双字、无符号字、无符号双字；{A/B/C}表示此处分别

使用 A、B、C 来参与构成不同的指令名，A[B]表示此处分别使用 A 和 AB 来参与构成不同的指令名；rd/rj/rk 为通用寄存器，fd 为浮点寄存器，vd/xd 为向量寄存器，si8/si9/si10/si11/si12/si14 为 8/9/10/11/12/14 位有符号立即数。

其中操作数包含立即数域的指令，其立即数的形式可能为 si8、si9、si10、si11、si12、si14，分别表示长度为 8、9、10、11、12、14 位的有符号数，可表示数值范围较小。以使用最频繁的普通访存 ld/st 指令为例，其立即数域 si12 可表示数值范围为 $-2048 \sim +2047$ 且无倍率放大功能，而 Arm 指令集中相应的 ldr/str 指令支持的立即数操作数，在常规模式下可表示数值范围为 $0 \sim 32760$ （无符号），宽度是 LoongArch 的 8 倍。

虽然从整体上看，指令的编码和功能设计属于架构的固有属性，在编译时无法改变，但考虑如前述的大尺寸栈帧数据读写等具体场景，仍有优化空间，可以采取尽量更充分地发挥指令能力。目前存取栈帧数据时构造地址的计算基准——帧指针 fp 位于栈帧的端侧，存储数据的空间基本都在其单侧方向，因此 LoongArch 的 ld/st 指令中立即数操作数的“有符号”特性并未被充分利用，若将地址计算的基准定位在数据区域的中间位置，便可把立即数的正值和负值区间都利用起来，避免浪费一半数值范围，详见后文第 3.4.2 小节。对于非栈帧的其他区域，同理也可尝试优化地址计算基准的选取策略。

2.2.3 大块连续内存初始化

在需要对大块连续内存进行初始化操作，例如新建一个特别大的结构体 struct 类型变量时，当前 LoongArch 上的指令实现模式为：整体上采用类似高级编程语言中 for 循环的结构，单轮循环中进行两次使用 store 指令零初始化一个寄存器大小的空间，循环的轮数由一个临时计数器变量控制，其初始值在循环开始之前计算出，每轮循环中自减 1，减至 0 时循环结束。

一个实际例子的汇编指令序列如下所示：

地址	指令
0xff7d250048	addi.d a0, fp, 24 ; a0 存放每轮循环操作的起始地址，最初为 fp+24
0xff7d25004c	lu12i.w a1, 0x1
0xff7d250050	ori a1, a1, 0xffc ; a1 = 0x1ffc = 8188，循环计数器初始值
0xff7d250054	st.d zero, a0, 0 ; 零初始化 8 个字节
0xff7d250058	st.d zero, a0, 8 ; 零初始化 8 个字节
0xff7d25005c	addi.d a1, a1, -1 ; 循环计数器自减 1
0xff7d250060	addi.d a0, a0, 16 ; 更新下一轮循环操作的起始地址
0xff7d250064	bne a1, zero, 0xff7d250054 ; 计数器未归零则跳转继续循环

由于 st 指令单次只能存取一个寄存器的大小，而要操作的内存区域特别大，

采用循环的方式实现是很自然的策略。但当前实现有两方面不足：

(1) 额外使用的用于控制循环轮次的计数器变量并非必需，每轮循环中需要使用一条指令（`addi.d`）更新其值，在循环轮数很大时会导致实际执行时的动态指令的数量也增加很多，也就意味着运行时间会更长。

可以考虑将其去除，直接使用 `st` 指令的访存地址与要操作的内存空间的边界位置的地址比较，来监控操作的进度，在到达边界后退出循环，如此可大幅减少相应动态指令的数量。设计改造后的指令序列如下所示：

<code>addi.d</code>	<code>a0, fp, 24</code>	; a0 存放每轮循环操作的起始地址，最初为 <code>fp + 24</code>
<code>lui2i.w</code>	<code>a1, 0x1f</code>	
<code>ori</code>	<code>a1, a1, 0xfd8</code>	
<code>add.d</code>	<code>a1, a1, fp</code>	; a1 存放循环操作的末尾地址 <code>fp + 131032</code>
<code>st.d</code>	<code>zero, a0, 0</code>	; 零初始化 8 个字节
<code>st.d</code>	<code>zero, a0, 8</code>	; 零初始化 8 个字节
<code>addi.d</code>	<code>a0, a0, 16</code>	; 更新下一轮循环操作的起始地址
<code>bne</code>	<code>a0, a1, -3 instructions</code>	; 未到达边界则跳转继续循环

若如此实现，静态指令数量仍为 8 条没有变化，但相应的动态指令总数则从原来的 $3 + 8188 * 5 = 40943$ 减少为了 $4 + 8188 * 4 = 32756$ ，降幅达约 20%，效果显著，且没有多使用寄存器等副作用。

(2) 每轮循环中 `st.d` 指令仅共操作了两个寄存器大小（两个双字，共 16 个字节）的空间，造成循环轮数很大。而每轮循环中除了关键的 `store` 操作外，还需更新计数器的值和下一轮操作的起始地址、判断是否要结束循环，循环轮数越多这些辅助操作执行的次数也就越多，即动态指令的总数量越多，运行时间越长。

这一现象本质上与循环展开（Loop Unrolling）编译优化技术针对的场景相符，因此可以采用相同的策略进行优化，即通过适当增大展开因子——每轮循环中 `store` 操作的个数，减少迭代轮数，从而减少控制开销，提高数据局部性和指令并行度，提高执行效率。增大展开因子可以减少动态指令的数量，但同时也会增加静态指令的数量（从而增大指令的体积，可能会影响到 CPU 缓存的命中率而又反过来导致性能下降），不过两者变化的趋势并不相同。

设要进行零初始化的内存空间总大小共相当于 N 个双字（`dword`），每轮循环中 `store` 操作的个数为 x 。由于 x 不一定能恰好整除 N ，在循环部分结束之后可能还需要额外的 `store` 操作处理剩余部分（数量为 $N \% x$ 个）。

基于前述 (1) 中优化改造后的机制，静态指令总数 $= 4 + x + 2 + N \% x = 6 + x + N \% x = 6 + (1 + N \% x / x) x < 6 + 2x$ ，动态指令总数 $= 4 + \text{循环次数 } n * (x + 2) + N \% x = 4 + N + 2n = 4 + N + 2N // x$ ，其中循环次数 $n = N // x$ 。可见，在 N

大小固定的情况下，随着 x 线性增大，静态指令总数成线性增长（比例系数介于 1 和 2 之间），而动态指令总数大致成反比例单调减小。增大 x 可在小幅增加静态指令数的同时大幅减少动态指令数，在权衡两方面重要性、给定综合效益具体评判规则的情况下，必可求得每轮循环 store 操作个数的一个最优值。

需要注意，每轮循环 store 操作个数 x （即展开因子）的增大不是没有限制的。

如前文 2.2.2 小节所述，指令中的立即数所能表示的数值范围是有限的。“st.d rd, rj, si12”指令和“addi.d rd, rj, si12”指令中的 si12 立即数对应范围为 $-2048 \sim +2047$ ，“bne rj, rd, offs16”指令中的 offs16 立即数由于计算时会带有逻辑左移两位的机制对应范围为 $-131072 \sim +131071$ ，而每条指令的大小均为 4 字节，因此可支持向前或后跳转 32768 条指令。这些数值限制了 x 的上界：

①每条 st.d 指令操作 8 个字节的空间，彼此连续而不重叠，则其立即数序列构成一个从 0 开始，以 8 为公差的等差数列，最大取值为不超过 2047 的最后一个 8 的倍数——2040，相应 x 的值即为该数列的项数 $2040 / 8 + 1 = 256$ 。

②addi.d 指令用来更新下一轮循环操作的起始地址。由于单个 store 操作是以 8 字节为单位，每轮循环操作的总量也必然是 8 的倍数。那么此处 addi.d 指令中立即数的上界也为不超过 2047 的最后一个 8 的倍数——2040，即每轮循环最多操作 2040 个字节的空间，相应 x 的值为 $2040 / 8 = 255$ 。

③bne 指令需要能够跳转跨过 addi.d 和所有 st.d 共 $x + 1$ 条指令，因此 x 最大可为 $32768 - 1 = 32767$ 。

综上，在前述（1）中优化改造后的指令实现机制下，由指令立即数表示范围有限造成的 x 的上界值为 255，要求的 x 的最优值自然处于 $[2, 255]$ 区间内。

进一步，若不局限于前述的指令实现机制，则还有一些其他可能：

①一个特殊的极端情形是，若 N 的值本就不超过 256，那么可以不采用循环，直接以全部展开的一连串 st.d 指令实现，如下所示：

addi.d	a0, fp, 24	; a0 存放操作的起始地址，fp + 24
st.d	zero, a0, 0	
st.d	zero, a0, 8	
st.d	zero, a0, 16	
.....		
st.d	zero, a0, (N-1)*8	

动态指令总数达到该情况下的最小值 $N + 1$ ，静态指令总数也为 $N + 1$ ，与采用循环但展开因子 x 取值较大的情形相当。

②前述各实现中 `st.d` 指令的立即数序列均是从 0 开始递增的，如 2.2.2 小节末尾所述，可以优化其地址计算基准的选取，从而利用起其中立即数的负值区间，将每轮循环可操作的空间大小上限翻倍，将 x 的上界由 256 变为 512。

如此实现还需要解决用于辅助更新下一轮循环 `store` 操作的起始地址的 `addi.d` 指令立即数范围造成的限制（`bne` 指令给出的限制 32767 远远触及不到），可以考虑多使用一个临时寄存器（若有）来将加法运算的指令从 `addi.d` 改为只使用寄存器操作数而不使用立即数的 `add.d` 指令，以 x 取最大值 512 为例，指令序列如下所示：

<code>addi.d</code>	<code>a0, fp, 2072</code>	; a0 存放每轮循环操作的地址计算基准，最初为 $fp+24+2048$
<code>lui2i.w</code>	<code>a1, 0x1e</code>	
<code>ori</code>	<code>a1, a1, 0x818</code>	
<code>add.d</code>	<code>a1, a1, fp</code>	; a1 存放地址计算基准的上界 $fp+24+(N-N\%512-256)*8$
<code>lui2i.w</code>	<code>t1, 0x1;</code>	; t1 每轮循环操作的空间大小， $x*8$
<code>st.d</code>	<code>zero, a0, -2048</code>	; 操作第一个 8 字节
.....		; 510 个连续的 <code>st.d</code> 操作，立即数从 -2010 增加到 2032
<code>st.d</code>	<code>zero, a0, 2040</code>	; 操作最后一个 8 字节
<code>add.d</code>	<code>a0, a0, t1</code>	; 更新下一轮循环操作的地址计算基准
<code>bne</code>	<code>a0, a1, -513 instructions</code>	; 未到达边界则跳转继续循环
.....		; $N\%512$ 个连续的 <code>st.d zero, a0, imm</code> ，处理剩余的部分

若 x 取值为 256~512 之间的值，需要调整 `a1` 中存放的上界值、`t1` 中存放的步长值（需再额外使用一条 `addi` 指令辅助计算）、`st.d` 的数量和立即数、`bne` 的立即数。按此方式实现时，指令数的计算规则也发生了变化，静态指令数 = $4 + 1$ （或 2）+ $x + 2 + N \% x = 7$ （或 8）+ $x + N \% x$ ，动态指令数 = $4 + 1$ （或 2）+ 循环次数 $n * (x + 2) + N \% x = 5$ （或 6）+ $N + 2N // x$ ，与前述（1）中优化改造后的指令实现机制相比，只是分别都增加了 1（或 2），趋势规律仍一致。

综上所述，除了 N 不超过 256 时的完全展开极端情形外，展开因子的取值范围为 $[2, 255]$ ，倘若允许多使用一个临时寄存器，则展开因子取值范围为 $[2, 512]$ 。

在允许的取值范围内确定展开因子的最优值，是循环展开这一技术的关键，是一个经典而复杂的课题，至今仍在被广泛研究，尚未有单一明确的结论。一方面，展开的综合效益评判规则并非一成不变，受编译器产品的客户需求等外在因素影响；另一方面，由于涉及因素众多，启发式方法很难准确建模求解，而且针对小且具体直接的场景使用过于复杂的算法拖慢编译速度也不一定值得。对于本文这一具体场景，还是通过对不同取值分别进行实验测试从而根据实际效果来选定最优解的迭代编译策略更为合适。

此外，若能使用 SIMD 类指令操作更宽的向量寄存器或使用 cache 维护类指令操作整个 cache 行来替代当前的 st 指令（最大只能操作 8 个字节），可以进一步提升效率。

2.2.4 小尺寸数据类型在寄存器中的表示

对于指令集架构而言，其定义的通用寄存器宽度一般是唯一的固定值，常为 32 或 64 比特，LoongArch64 架构下为 64 比特（8 字节），而高级编程语言和编译器内部支持的数据类型一般有多种，如 bool、byte、short、int、long、float、double 等，其尺寸不一，以字节为单位，有 1、2、4、8 等多个可能值。于是，那些尺寸小于寄存器宽度的数据在需要利用寄存器参与运算时怎样在其中保存，以确保在后续参与运算或被存储时保证值的准确，便是一个需要编译器解决的问题。

前述用例程序中便有一些场景，展现了当前 LoongArch 架构上的 RyuJIT 编译器对此问题解决机制存在的不足。两个相关的实际指令序列片段如下所示：

指令序列 1		指令序列 2	
ld.w	a0, t1, -144	and	a0, a0, a1
slli.w	ra, a0, 0	slli.w	a0, a0, 0
beqz	ra, 0xff7d660634	bstrpick.d	a0, a0, 7, 0

指令序列 1 中，ld.w 指令访存读取了一个字（4 字节）的数据写入通用寄存器 a0，slli.w 指令将 a0 中数据逻辑左移零位后写入寄存器 ra，beqz 指令判断 ra 寄存器中的值，若等于 0 则跳转到目标地址。其中 slli.w 指令左移零位操作看起来毫无意义，是冗余指令，但实际上左移运算的操作数只是源寄存器 a0 中的低 32 位，而后将结果写入 64 位的目标寄存器 ra 时有一个附加效果——符号扩展。由于后面的 beqz 指令判断值是否为 0 时看的是整个源寄存器 64 位的数据，为了保证其高 32 位不是脏数据，造成结果出错，才添加了这条 slli.w 指令。不过，对于这个例子，前面的 ld.w 指令其实在将读到的数据写入目标寄存器时 a0 时已经同时做了符号扩展，所以整个 a0 寄存器的 64 位比特没有问题，slli.w 指令确实是冗余的。

指令序列 2 中，and 指令将通用寄存器 a0 和 a1 中的数据进行按位逻辑与运算，结果写入 a0，slli.w 指令将 a0 中的数据逻辑左移零位后再写入 a0，bstrpick.d 指令提取 a0 中的 [7:0] 位零扩展至 64 位后再写入 a0。与前一个例子类似，中间的 slli.w 指令左移零位操作看起来是冗余的，但实际上设计目的是利用其符号扩展效果将只有 32 位的实际有效数据在 64 位的寄存器中表示的更准确，确保其高 32 位不是脏数据。不过，对于这个例子，slli.w 指令也确实是冗余的，因为后续

的 `bstrpick.d` 指令根本用不上 `a0` 寄存器的高 32 位内容。

这两个例子表现相似，但冗余 `slli.w` 指令产生的根源在编译器内部对应的是两种不同的情形：指令序列 1 对应的是为比较跳转（`JCMP`）节点生成指令时，若实际有效数据尺寸为 4 字节则利用 `slli.w` 指令做前置处理，保证自身操作的正确性；指令序列 2 对应的是为与（`AND`）、或（`OR`）、异或（`XOR`）三种逻辑运算节点生成时若实际有效数据尺寸为 4 字节则利用 `slli.w` 指令做善后工作，为后续节点提供准确性保证。

追根究底，这两种情形实现的本质目的相同，都是为了消除尺寸小于寄存器宽度的实际有效数据时在寄存器中周转时寄存器高位可能存在的脏数据，以免其影响程序逻辑的正确性。但实现的有些粗糙，该操作是统一添加而非按需采用的，即使实际并无必要也会同样生成指令，造成冗余浪费。

另外，实际上 `LoongArch64` 架构的基础整数（定点）指令，在操作不满 64 位的数据时基本全都在完成基础功能时自带了符号扩展或零扩展效果，尽量保证寄存器整体表示数值的可靠性，出现脏数据的可能性很小。

因此，专门使用指令清除寄存器高位脏数据最好只在少数确有需求的情况下才实现，整体上可以考虑采用如下机制：从各种数据第一次进入寄存器的源头，便开始按需添加符号扩展或零扩展操作，维护所在寄存器全部位域整体表示的准确性，并在后续流转过程中一直维持。

2.2.5 其他

其他潜在待优化场景还有很多，例如有些判断数组下标是否越界的场景，受限于所需条件转移指令的立即数支持范围不够大，覆盖不了所需偏移量，当前连续使用了两条转移指令完成常规逻辑和异常处理逻辑的执行路径隔离，而实际上可以在一条算术运算指令的辅助下，仅使用一条转移指令实现，减少跳转需要；很多函数调用的地方无条件跳转指令选用了基于绝对地址的 `jirl` 指令，而绝对地址的构造较为繁琐，对于那些偏移量不很大的情形，可以改用基于相对 `PC` 偏移的 `bl` 指令；有些地方寄存器复用不充分，相同数值多次重复计算……限于篇幅此处不再详述。

2.3 本章小结

`LoongArch` 架构上的 `.NET` 底层 `RyuJIT` 编译器在基本适配完成后尚未系统性地开展优化，其编译生成指令的质量有待优化提升。

本文利用 `RyuJIT` 编译器的反汇编功能对源码库内嵌的测试用例集进行了编

译生成静态指令数量的统计，并对比了 LoongArch 和 Arm 上统计结果数据的差异，发现用例程序在 LoongArch 架构上的对应指令数普遍要比在 Arm 架构上更多，仅极少量用例程序 LoongArch 有优势。分析其中 LoongArch 指令数量劣势较大的一些典型用例程序的指令序列，结合架构特点，对访存寻址方式、大块连续内存初始化、小尺寸数据类型在寄存器中的表示、转移指令的利用等场景或问题展开探究，提出了一些较为具体的优化构想，为未来继续改进完善 RyuJIT 编译器提供了参考。

第 3 章 缓冲区溢出防护机制失效漏洞分析与修复

LoongArch 平台上 .NET 底层 RyuJIT 编译器的原有实现, 在部分特殊场景下, 存在函数返回地址被缓冲区溢出篡改而防御机制不起效、无法及时检测到的安全漏洞。本章阐述该漏洞的发现、分析与修复相关工作, 其中 3.1 节介绍该漏洞的相关背景, 3.2 节通过验证实验将其暴露展现, 并分析其产生原因, 3.3 节介绍修复漏洞所做工作, 3.4 节给出修复的结果, 并拓展延伸提出进一步优化的思路, 3.5 节对本章内容作简要总结。

3.1 背景介绍

3.1.1 缓冲区溢出

一直以来, 缓冲区溢出 (Buffer Overflow) 都是计算机安全领域最为常见的安全漏洞形式之一, 广泛存在于各类操作系统、应用软件和网络协议中, 是黑客攻击和恶意软件利用的重要入口。其宏观原理相当简单——预先分配的固定大小的缓冲区被写入了超额的数据以致于相邻内存区域被意外覆盖/篡改, 但利用其进行实际攻击/防御的实现方式复杂而又多样, 导致其在研究和实践中经久不衰。

最早的缓冲区溢出攻击发生于 1988 年 11 月, 始作俑者是康奈尔大学的一名研究生——罗伯特·塔潘·莫里斯 (Robert Tappan Morris), 他编写并释放传播的仅 99 行代码的 Morris 蠕虫程序, 利用了 UNIX 系统中存在的多个漏洞, 其中便包括 Finger 程序未限制输入长度导致的缓冲区溢出漏洞。尽管莫里斯的本意只是为了满足自己的好奇心——统计互联网的规模 (共接入了多少台设备), 而非真的恶意攻击, 但最终此次失控的“攻击”在短短 12 小时内便导致了超过六千台计算机 (约占当时接入互联网计算机总数的 10%) 瘫痪或半瘫痪, 直接造成了近亿美元的经济损失。Morris 蠕虫事件引起了人们的广泛关注, 也使得缓冲区溢出漏洞开始成为了计算机安全领域的一个重要研究课题。

缓冲区溢出的根源在于, 涉及到的相关程序一般是采用开发自由度大、性能高但不安全的 C 语言编写的, 而 C 语言中缺乏类型安全保障机制, 在程序向缓冲区试图写入超出其预分配的大小的过量数据时, 不会自动进行数组边界检查、及时发现问题并加以限制, 从而导致数据溢出并覆盖到相邻内存区域中保存的变量、函数指针、寄存器等内容, 在程序后续读取这些内容进行解释或执行时, 错误的值便会导致程序的正常执行流程被破坏, 出现崩溃、错误跳转等意外后果。

利用缓冲区溢出的攻击形式中，最常见的是栈溢出攻击（Stack Smashing Attack），通过在函数调用栈上构造溢出以执行恶意代码。如图 3.1 所示，攻击者精心设计特定的输入数据，使得其在被写入缓冲区并发生溢出后可以精准篡改掉栈上保存的函数返回地址，使其指向构造好的恶意代码（通常为 shellcode^[12]，用于启动一个 shell 进而执行其他命令），在当前函数执行完毕跳转返回时便会脱离原有逻辑转而进入恶意代码，从而劫持控制流，实现攻击的目的。

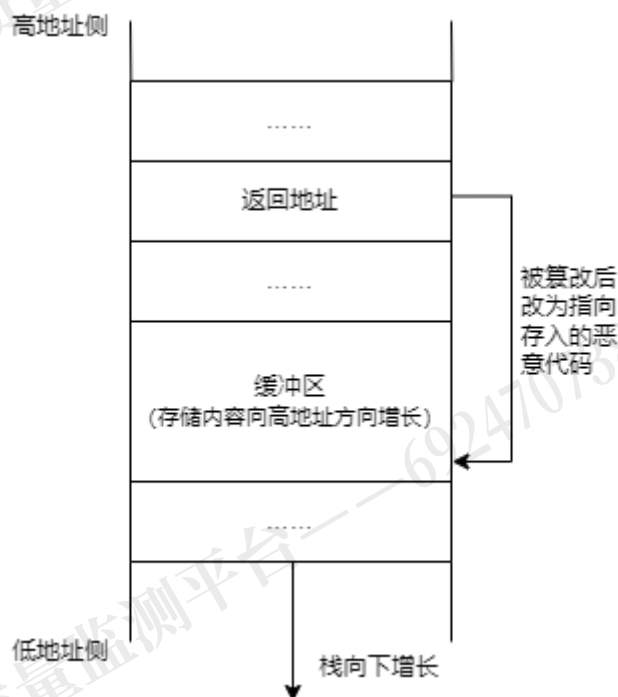


图 3.1 堆栈溢出攻击原理示意

堆栈溢出攻击所利用的缓冲区，通常是字符数组，一般只需要在程序源代码中声明定义便会在栈上自动分配内存块，如使用语句“`char buffer[10];`”，但除此之外还有一种允许程序员显式分配的操作，在 C 语言中由 `stdlib.h` 中的 `alloca` 函数^[13]实现，可以在运行时在访问速度较快的栈（而非更常见的堆）上根据需要动态分配内存块，且大小可变，在退出调用 `alloca` 的函数时内存块又会被自动释放，无需手动 `free`。但实际上 `alloca` 的使用也很容易出错，对程序员的水平要求较高，容易导致栈溢出，没有在各种系统中受到普遍支持，可移植性较差。不过在最主流的 GNU 系列系统中，`alloca` 通常是可用的（但不受推荐）。

在 .NET 生态中，`alloca` 技术也得到了支持，在中间语言 IL 中对应为 `localloc`^[14] 指令，在 C# 语言中通过 `stackalloc`^[15] 关键字使用，如“`stackalloc int[10]`”。

3.1.2 StackGuard 防护技术

根据缓冲区溢出的原理，很自然可以想到一种修复漏洞的方案——在程序源代码中补充添加上缺少的边界检查判断处理逻辑即可。单个缓冲区溢出漏洞如此修补确实很容易、很有效，但这种漏洞是广泛存在、难以计数的，要评审所有（包括已有的和未来的）程序的代码、找出所有漏洞并针对性地逐一完成修复并不现实，因此更加通用的应对方法仍然很有必要。

几十年来，研究者们已经提出了许多缓冲区溢出漏洞的防范措施，如改用类型安全的编程语言（如 C#、Java 等）或库函数（如 `strncpy`、`snprintf` 等）编写程序、地址空间布局随机化（Address Space Layout Randomization, ASLR）^[16] 技术、StackGuard 栈保护技术…… 其中 StackGuard 技术是对编译器的扩展，尽管并不完美，但简单而有效的高“性价比”使得其获得了广泛应用，在主流的现代编译器如 GCC^[17,18]、LLVM/Clang^[19]、MSVC^[20] 中都有相关实现，并且不断发展衍生出许多后续版本^[21]。

StackGuard 技术最初由 Crispian Cowan 等于 1998 年提出^[22]，是第一种针对栈溢出攻击的防御措施，如图 3.2 所示，基本原理是在缓冲区和保存的函数返回地址之间插入一个特殊字（称为 canary），当缓冲区溢出时，在其能够覆盖返回地址之前，必然会先经过 canary，将其内容覆盖。这样只要在函数执行结束读取返回地址之前校验 canary 的值是否发生改变，便可判断是否发生了缓冲区溢出，进而根据需要采取应对措施，从而实现了溢出的检测和防护。

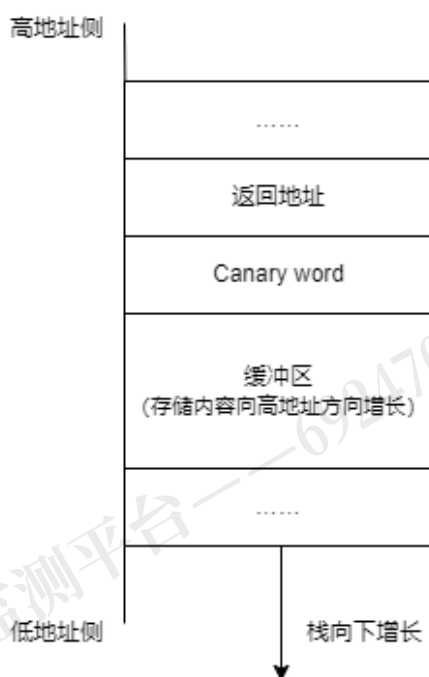


图 3.2 StackGuard 防护技术原理示意

StackGuard 技术不需要更改程序的源代码,与原有操作系统和库二进制兼容,适用范围广,可有效地应对大多数栈溢出攻击,此外插入和检查 canary 值的性能开销相较于其带来的好处而言也非常小(通常小于 1%^[23]),而且还可以根据需求采用不同的变种^[21,22]以进一步平衡安全性和性能成本。

在 .NET 中,底层的 RyuJIT 编译器内部也实现了 StackGuard 技术,为上层 C# 等语言的类型安全提供了保障。当在 C# 应用程序中使用到栈上缓冲区时,会自动启用 StackGuard 机制,如果检测到缓冲区溢出,则会尽快终止进程,以最大程度地降低执行恶意代码的可能性^[15]。

3.2 漏洞表现与分析

前文提到,alloca/localloc 是一种特殊的栈上内存分配操作,虽然在许多系统中是可用的但并不受推荐,因而应用其分配缓冲区也就成了一种少见的场景。此外,使用 alloca/localloc 分配的缓冲区由于大小可变,在函数栈帧上的位置布局一般也要与常规的数组变量缓冲区有所分隔。要应对这种特殊场景下的缓冲区溢出风险,StackGuard 防护机制的实现往往也需要进行针对性调整。

在 LoongArch 平台上,.NET 底层 RyuJIT 编译器的原有实现没有适配好这一特殊场景,使得 localloc 缓冲区的溢出有比较大的机会篡改掉函数返回地址,而 StackGuard 机制却无法按照预期及时发现检测到。

3.2.1 漏洞的表现

为暴露展现该漏洞,设计简单 C# 程序进行实验验证,代码如图 3.3 所示。

在该例程中,主函数 Main() 调用子函数 Overflow(),之后打印一行字符串,指示本函数执行结束;子函数 Overflow() 中使用 stackalloc 关键字在栈上动态申请分配大小为一个字节的数组缓冲区,之后通过循环向其中写入内容,最后也打印一行字符串指示本函数执行结束。

子函数 Overflow() 中,通过使用指针操作,在向数组缓冲区写入内容时绕过了常规的自动边界检查机制,同时也没有手动进行边界检查,于是当索引变量 i 的值超过数组声明的大小(≥ 1)时便会产生溢出。

设置代码中数组索引变量 i 的自增上界为不同数值,分别在 LoongArch64 平台和适配更为充分且同为 RISC 风格的 Arm64 平台上使用相同版本的 .NET runtime (CoreCLR) 构建运行该程序,结果如表 3.1 所示。

```
class Program {
    static void Main(string[] args) {
        Overflow();
        Console.WriteLine("Main 函数末尾");
    }

    [MethodImplAttribute(MethodImplOptions.NoInlining)]
    static unsafe void Overflow()
    {
        byte* ptr = stackalloc byte[1]; // 在栈上分配缓冲区，并使用指针承接
        for (int i = 0; i < 32; ++i)      // 通过调整变量 i 的上界控制缓冲区的溢出
        {
            ptr[i] = 0;
        }
        Console.WriteLine("Overflow 函数末尾");
    }
}
```

图 3.3 缓冲区溢出安全漏洞验证程序

表 3.1 漏洞验证程序中变量 i 的上边界设置为不同值时的运行结果

情况编号	变量 i 的上边界值	LoongArch64 平台运行结果 (终端输出内容)	Arm64 平台运行结果 (终端输出内容)
①	1	Overflow 函数末尾 Main 函数末尾	Overflow 函数末尾 Main 函数末尾
②	16	Overflow 函数末尾 Main 函数末尾	Overflow 函数末尾 Main 函数末尾
③	32	Overflow 函数末尾 段错误	Overflow 函数末尾 已放弃
④	48	Overflow 函数末尾 已放弃	(死循环无法结束)

①当 i 的自增上界设置为 1（最大有效值为 0）时，循环过程中始终没有超出数组索引范围，即没有发生溢出，两平台上均打印出了两函数的结束指示字符串，表明程序正常结束。

②当 i 的自增上界设置为 16（最大有效值为 15）时，循环过程中其值超出了数组索引范围，发生了溢出，但在两平台上程序仍正常结束，这实际是因为栈

指针 (Stack Pointer, SP) 的 16 字节地址对齐需要导致了缓冲区上方存在空槽 (或称为填充, 详见下一小节分析), 因此虽然发生溢出但覆盖的区域无关紧要, 没有不良后果。

③当 i 的自增上界设置为 32 (最大有效值为 31) 时, 缓冲区溢出范围进一步扩大, 两平台上程序运行均异常中止, 但表现有所不同。在打印出子函数的结束指示字符串后, LoongArch64 上报段错误, 表明程序中触及了非法内存地址, 属于意外错误; 而 Arm64 上程序放弃继续执行, 是进入了预设的异常处理流程。

④当 i 的自增上界设置为 48 (最大有效值为 47) 时, 缓冲区溢出范围继续扩大, 两平台上程序运行均异常中止但表现不同, 且与前一种情况又有所区别。LoongArch64 上程序放弃继续执行, 进入了预设的异常处理流程; 而 Arm64 上程序进入死循环无法退出。

对于同一 C# 程序, 在发生缓冲区溢出时 LoongArch64 和 Arm64 平台上出现了不同的表现, 其中 LoongArch64 在情况③的输出和 Arm64 在情况④的输出表明其没有做出理想的有效防护。前一小节提到, .NET 底层 RyuJIT 编译器内实现了 StackGuard 技术, 在检测到缓冲区溢出时会尽快终止进程, 情况③中 Arm64 和情况④中 LoongArch64 上的输出也证明了该机制的有效性。那么两种防护“失效”情况原因何在?

3.2.2 漏洞的根源分析

实现 StackGuard 技术在编译器中实际所需实现的内容包括^[18]:

①确定 canary 字在栈上的位置——特定于体系结构, 因为它需要知道堆栈的布局方式;

②为 canary 字分配空间——特定于体系结构, 因为它最终对应的相关指令属于特定于架构的函数序言 (prologue);

③初始化 canary 字的值——与体系结构无关;

④校验 canary 字的值是否发生改变——与体系结构无关;

⑤释放 canary 字所占的空间——特定于体系结构, 因为它最终对应的相关指令属于特定于架构的函数尾声 (epilogue);

⑥在校验失败时报告相关上下文信息——与体系结构无关。

前一小节验证实验中, 对于 malloc 特殊缓冲区溢出场景 StackGuard 机制在 LoongArch64 和 Arm64 两种架构上出现了不同的结果, 表明体系结构相关的环节, 即上述①、②、⑤之中, 很可能存在问题。其中环节①的栈帧布局是防护机制的本质所在, 环节②和⑤只是①的落地实现, 因此需要分析两架构平台上的函数栈帧布局设计。

在 LoongArch64 平台上的 RyuJIT 编译器的原有实现中，函数栈帧布局设计如图 3.4 所示，其中 GS Cookie 即前述 StackGuard 机制中的 canary 字。可见当 `alloca` 缓冲区发生向高地址侧的溢出覆盖时，保存的帧指针 FP (Frame Pointer) 和返回地址 RA (Return Address) 首当其冲，用于检测溢出的 GS Cookie 反而在距离更远的高地址位置，需要溢出更多才能被更晚地覆盖到，如此便出现了漏洞，即将溢出程度控制在中间范围时，就可以做到篡改返回地址而不被 StackGuard 机制发现，从而劫持程序逻辑、夺取控制流、实现攻击。

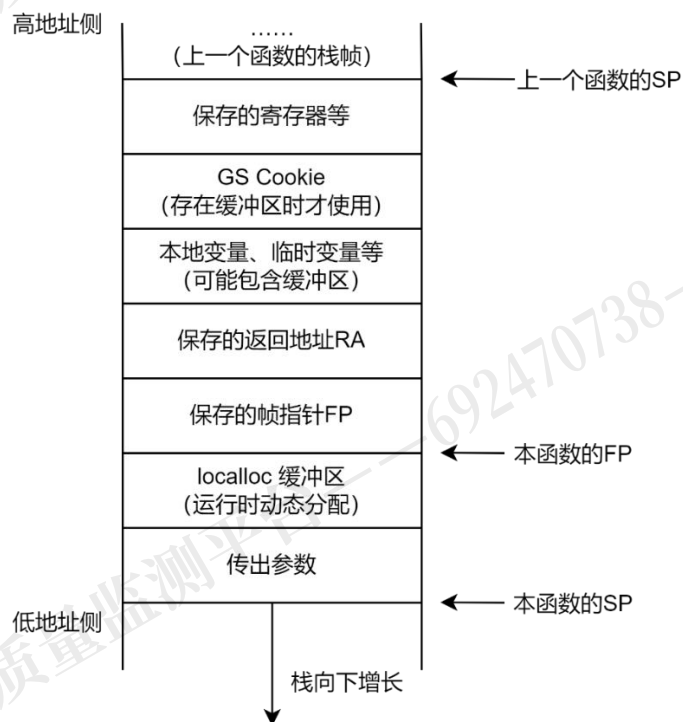


图 3.4 LoongArch64 上 RyuJIT 编译器原始实现的栈帧布局

特别地，对于上一小节的验证实验，例程中子函数 `Overflow()` 对应的栈帧布局情况可精简为如图 3.5 所示。

在实验的情况①和②中，缓冲区没有溢出覆盖到栈帧上有效数据，因此程序正常结束。情况③中，循环变量 `i` 最大值可达 31，循环结束时保存的帧指针 FP 和返回地址 RA 区域 (共 16 字节) 已被全部覆盖为 0，而更高地址处的 GS Cookie 没有被触及，因此在子函数执行结束时 StackGuard 机制校验 GS Cookie 的值不会发现异常，但跳转返回主函数时读取到的返回地址数值为 0，这一非法地址自然会引发段错误。情况④中，进一步扩大的溢出范围导致 GS Cookie 也被覆盖为 0，因此在子函数执行结束时 StackGuard 机制校验失败，程序会转而进入预设的异常处理流程，尽快中止放弃。

各架构平台的 RyuJIT 编译器实现处于统一的大框架之下，各自的函数栈帧布局设计整体上也比较相似。对于不含 `alloca` 特殊缓冲区的函数，Arm64 平台

采用与前述 LoongArch64 基本一致的栈帧布局，而对于含有 `alloca` 缓冲区的函数则做了单独的例外设计，将帧指针和返回地址改存到了栈帧的高地址一端。验证实验例程中子函数 `Overflow()` 对应的布局如图 3.6 所示。

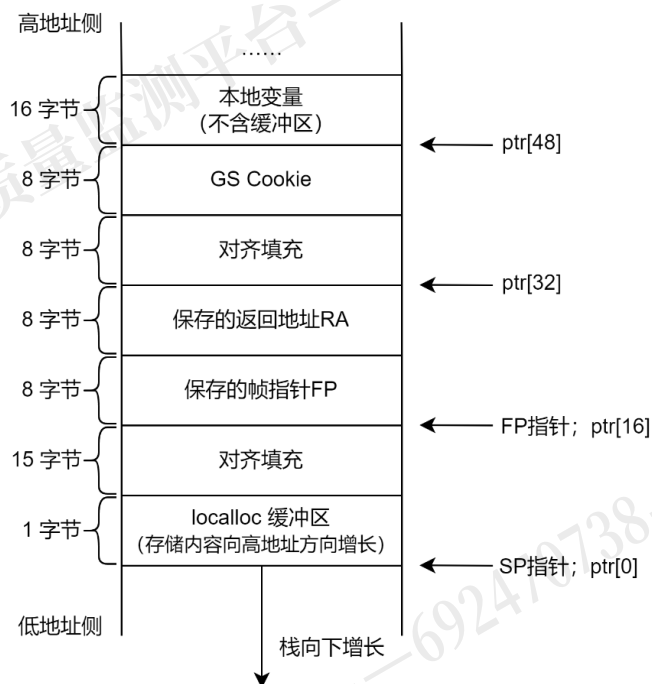


图 3.5 LoongArch64 上例程子函数对应的原栈帧布局

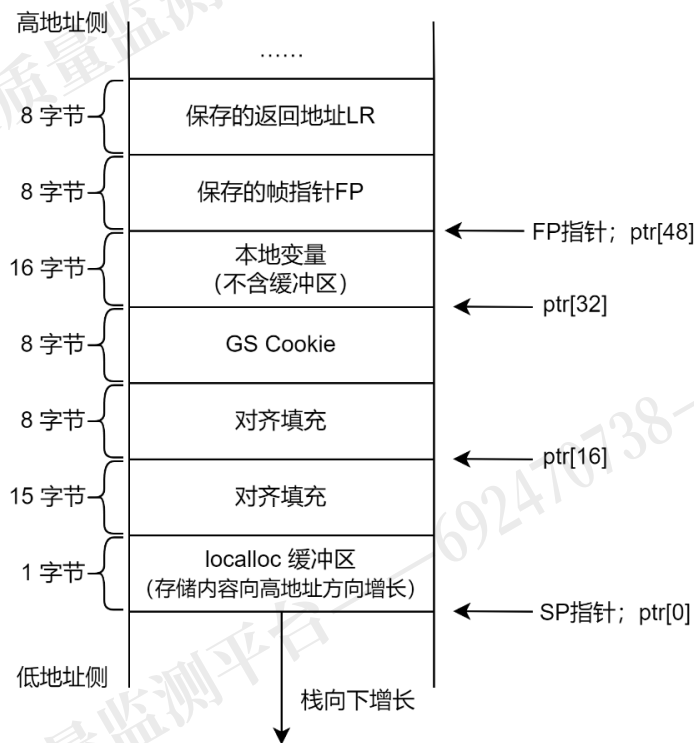


图 3.6 Arm64 上例程子函数对应的栈帧布局

在实验的情况①和②中，与 LoongArch64 一样，缓冲区没有溢出覆盖到栈帧上有效数据，因此程序正常结束。情况③中，索引变量 i 达到最大值 31 时，GS Cookie 也被覆盖为 0，而保存的帧指针 FP 和返回地址 LR 由于所处位置地址更高未被触及，在子函数执行结束时 StackGuard 机制校验失败，程序转而进入预设的异常处理流程，及时中止放弃。情况④中，进一步的溢出覆盖到了本地变量区域，其中包括控制循环次数的数组索引变量 i ， i 的值被覆盖为 0 后，赋值操作的位置也即被重置，随着 i 重新开始自增，缓冲区又重新开始溢出，直至再次将变量 i 覆盖重置，如此反复，无法退出循环。

通过以上分析，可见正是由于 LoongArch64 和 Arm64 两架构平台上的函数栈帧布局设计不同，导致 `localloc` 特殊缓冲区溢出时覆盖的内容也不同，才使得运行结果出现差异。其中影响 StackGuard 机制发挥作用的主要是缓冲区、GS Cookie（canary 字）、和返回地址三者间的相对位置关系，相比于 Arm64，LoongArch64 在该场景下的原有实现没有对其做出合理的排布。

缓冲区、GS Cookie 和返回地址三者在栈帧布局中的位置并非完全自由：`localloc` 缓冲区由于是运行时动态分配且大小可变，为减小对栈上其他数据的影响只能置于栈增长方向的端侧，与本地变量等内容隔开；GS Cookie 要防护栈帧中的所有缓冲区，而本地变量中也可能包含，因此在常规和 `localloc` 缓冲区同时存在时也只能置于比前者地址更高的位置；只有返回地址的位置没有特殊限制，可根据平台架构自身情况自由选择。

因此 LoongArch64 上的原有实现的问题根源就在于，在此特殊场景下为返回地址 RA（和帧指针 FP）在栈帧上安排的保存位置不合理，成为 StackGuard 机制实现的缺陷。而 Arm64 平台上虽也未达到最理想的防护效果，但该机制本身并无问题，符合其设计预期，覆盖变量导致的死循环已经超出其应对范围。

需要说明的是，由于前述验证实验程序设计得较为简单，其中的缓冲区溢出仅仅是将溢出区域覆盖为 0，而非有效的内存地址，才导致 LoongArch64 上的运行结果是报段错误、程序直接终止。若精心设计更为复杂、专业的攻击，如 3.1.1 节所述完全可以实现跳转执行恶意代码，但本实验目的仅为验证漏洞的存在，且安全攻防超出研究范围，故不进行拓展深入。

3.3 漏洞修复

如前所述，该漏洞的根源在于，在 `localloc` 场景下栈帧布局中的返回地址 RA（和帧指针 FP）保存位置不当，那么显而易见的修复方案便是对其进行调整，而且作为对比的 Arm64 架构也提供了参考。

落实到编译器的源码，实际修复工作包括两方面：调整函数栈帧布局中相关内容的排布位置（相对于栈帧端侧的偏移量）、同步调整架构相关的函数序言/尾声指令生成逻辑。

3.3.1 调整栈帧布局

参考 Arm64 的方案，对于存在 `localloc` 特殊缓冲区的函数，在 LoongArch64 上新添加使用例外布局，如图 3.7 所示，将帧指针 FP/返回地址 RA 寄存器组改存至栈帧高地址一端，使其和缓冲区（包括常规和 `localloc`）分列于 GS Cookie 两侧，以保障 StackGuard 检测机制的有效性。

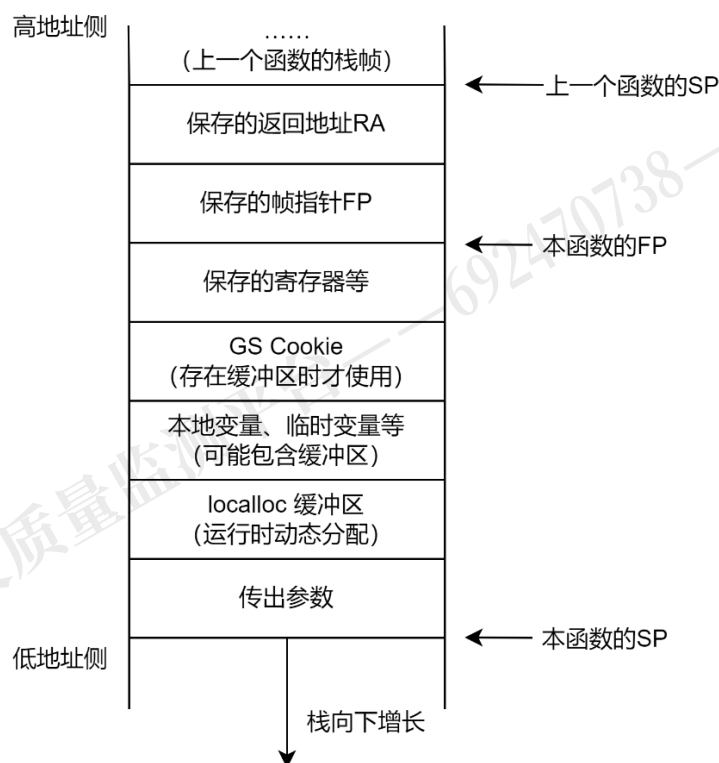


图 3.7 LoongArch64 上 RyuJIT 编译器新增的栈帧布局

在 RyuJIT 编译器源码中，栈帧布局主要在 `lclvars.cpp` 文件中的 `Compiler::lvaAssignVirtualFrameOffsetsToLocals()` 函数中实现，其原有实现和修复版本的主体逻辑如图 3.8 所示。其中从上到下逐步分配帧内偏移量本质即为各元素安排位置。

修复版本的所做的改动主要是：①在开头增加了对 `localloc` 场景的识别，以选择布局方案；②将为 FP/RA 分配帧内偏移量的操作，由原来的一处固定位置实现，改为在两处位置根据确定的布局方案选择性实现。

除上述编译器自动识别 `localloc` 场景从而自动切换采用新栈帧布局外，修复

工作中还实现了通过在外部设置环境变量而手动进行选择、管理（如强制使用、禁用）的机制，增加了灵活性。

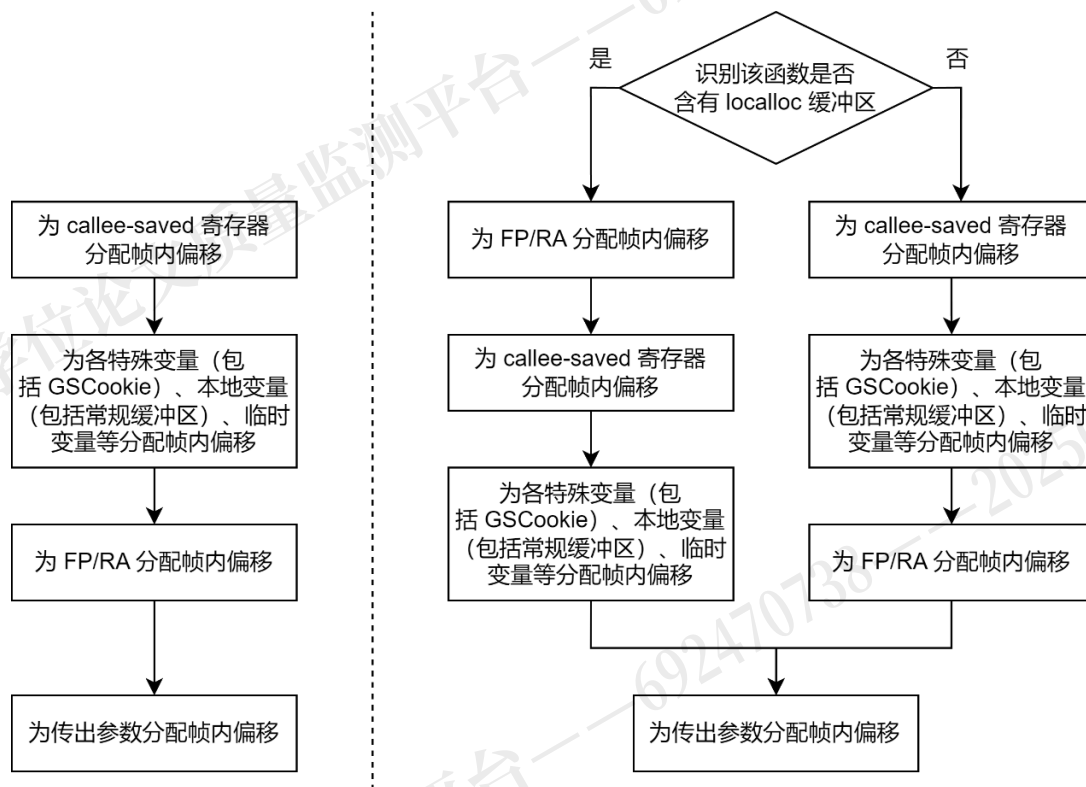


图 3.8 LoongArch64 上实现栈帧布局的函数原有实现（左）和修复版本（右）的主体逻辑

3.3.2 调整函数序言/尾声指令生成逻辑

函数的序言和尾声是栈帧布局的落地实现，也需对其进行同步改动。修复方案本质上是调整了帧指针 FP/返回地址 RA 寄存器组的保存位置，而栈上其他内容并未改动，因此只需同步调整序言/尾声指令生成逻辑中与 FP/RA 寄存器的保存/恢复相关的部分。

在 RyuJIT 编译器源码中，FP/RA 寄存器与其他被调用者保存寄存器（callee-saved registers）一并处理，涉及的内部函数主要是对应于程序中常规函数的 `CodeGen::genPushCalleeSavedRegisters()`、`CodeGen::genPopCalleeSavedRegisters()`，和对应于异常处理小函数 `Funclet`（一种优化的异常处理机制）的 `CodeGen::genCaptureFuncletPrologEpilogInfo()`、`CodeGen::genFuncletProlog()`、`CodeGen::genFuncletEpilog()`，以及其他若干辅助工具函数。

`CodeGen::gen[Push/Pop]CalleeSavedRegisters()` 函数对的两端分别对应于序言和尾声：前者（Push）主要用于开辟栈帧空间、保存寄存器；后者（Pop）主要用于恢复寄存器、释放栈帧。两者的逻辑在宏观上互为逆向操作，在本质上相差

不大，此处只以前者（Push）为例介绍。其原有实现和修复版本的主体逻辑分别如图 3.9 和图 3.10 所示，修复版本所做的改动主要是：①按照栈帧布局的调整，增加了对应的细分情况及相应处理；②重构了流程中各操作的整体组织结构，使其更为对称、清晰。

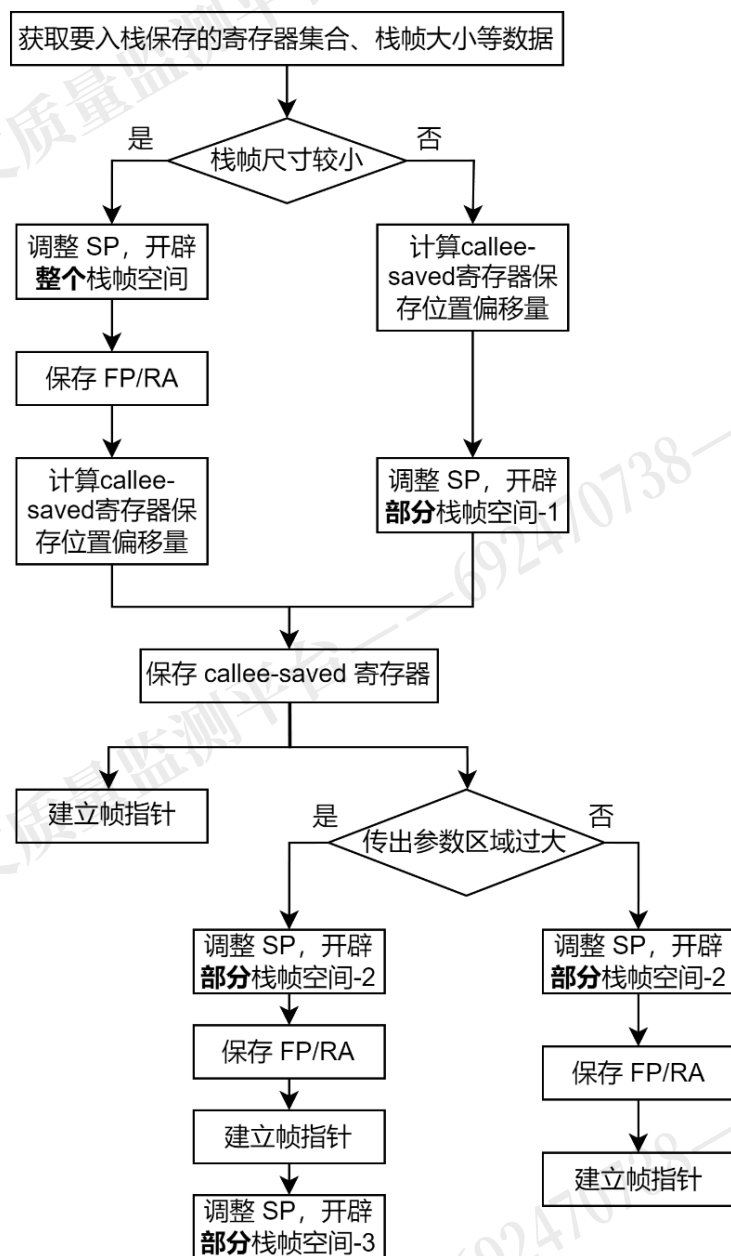


图 3.9 CodeGen::genPushCalleeSavedRegisters() 函数的原有实现主体逻辑

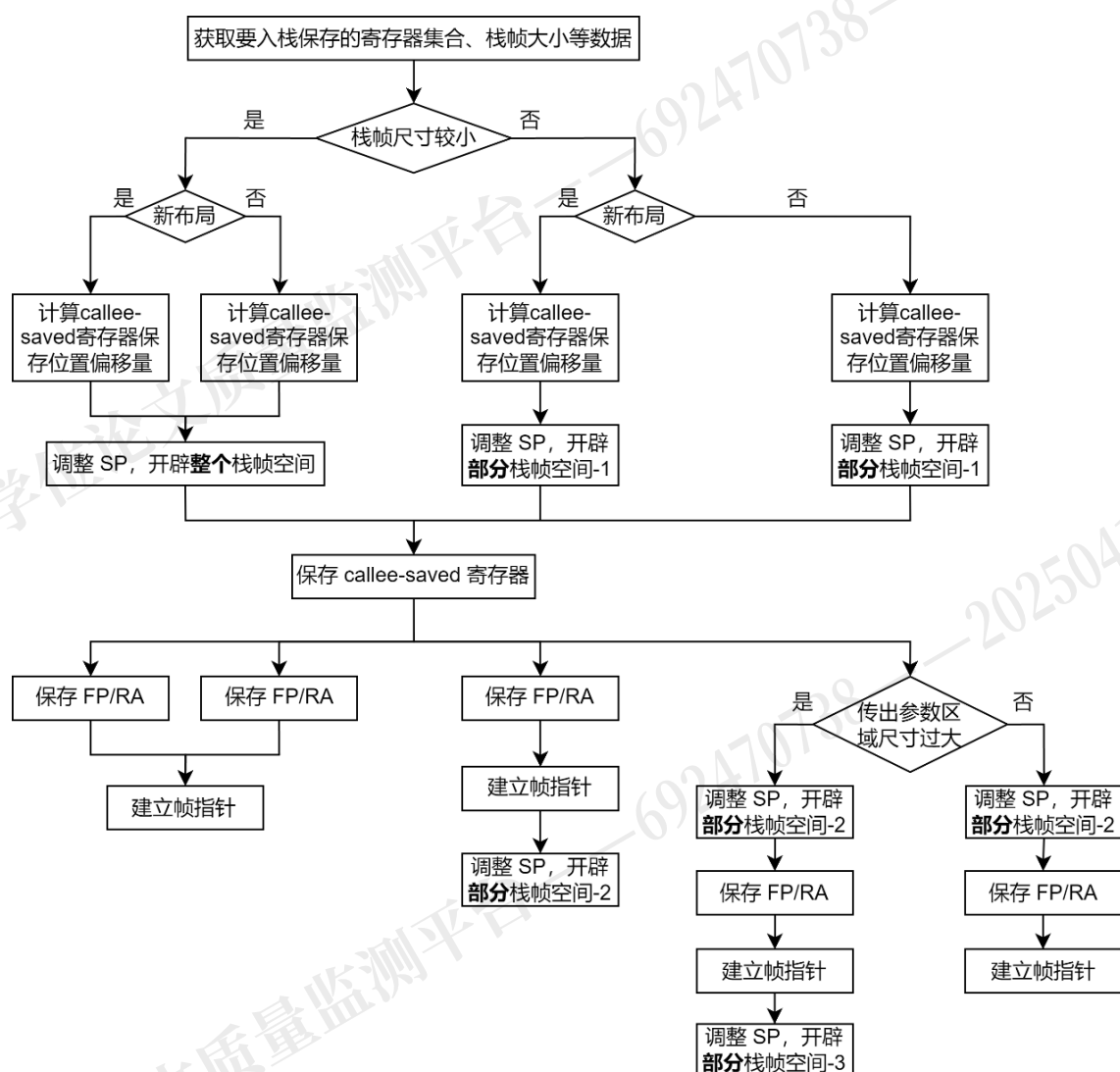


图 3.10 CodeGen::genPushCalleeSavedRegisters() 函数的修复版本主体逻辑

类似地, CodeGen::genFunclet[Prolog()/Epilog]() 函数对应的两者, 分别对应于异常处理小函数 Funclet 的序言和尾声, 也是互为逆向的操作, 分别用于建立和释放栈帧, CodeGen::genCaptureFuncletPrologEpilogInfo() 函数为其预处理一些所需数据。Funclet 和所属常规函数相对应, 但其栈帧更小, 其中内容有相同也有不同, 但总体上序言/尾声所完成的任务和实现逻辑都与后者较为相似, 且相对更简单一些, 此处不再赘述介绍。

3.4 修复结果与优化拓展

3.4.1 修复结果

完成修复后重新编译构建 RyuJIT 编译器 (.NET runtime) 本身, 使用 3.2 节中的实验例程重新进行测试, 此时各情形的程序运行结果均与 Arm64 平台一致

(参见表 3.1)，StackGuard 机制正常发挥作用。

使用 .NET runtime 源码库内置功能性测试集进行回归验证全面测试，结果与修复前原有版本相比通过率完全相同，未引入新的测试用例失败。

综上所述，对 RyuJIT 编译器所做的调整改动成功修复了漏洞，符合预期。

3.4.2 进一步优化的思路拓展

前述修复方案整体上是借鉴 Arm64 架构的已有实现，成功堵上了漏洞，算是解决了眼前问题。但其中增加的场景识别、判断选择等步骤，也增加了 RyuJIT 编译器自身逻辑的复杂度，一定程度上也影响实时编译效率，进一步考虑完全舍弃原有栈帧布局、只保留新增的布局设计作为通用布局以实现简化的可行性。

首先研究适配充分的 Arm64 架构采用该方案的原因。结合设计文档^[24]分析 Arm64 架构指令集^[25]中与此处设计相关的一些指令的特性，主要是有多种变体形式的 load/store 类指令。这些指令在此主要使用基于立即数的偏移寻址模式，而其中的立即数域支持的偏移范围并不相同，常用的 ldr/str 指令无符号偏移范围是 0~32760（且需为 8 的倍数），预索引/后索引偏移范围是 -256~255；ldp/stp 指令预索引/后索引/有符号偏移范围是 -512~504（且需为 8 的倍数）。可见 ldr/str/ldp/stp 指令支持的有符号立即数范围过小，在并不算罕见的栈帧尺寸稍大一些的场景下难以高效完成访存操作，而 ldr/str 指令支持的无符号立即数范围可用性尚可。因此，常规栈帧布局将多数情况下偏移寻址的基准——帧指针 FP 保存在栈帧的低地址一侧，能够更充分地利用访存指令的寻址能力，发挥编码优势，而“例外”布局只是补足安全性的硬性需要，性能更差故而不作为主力方案。

LoongArch64 指令集中 load/store 类指令特性与 Arm64 不同，常用的 ld/st 指令中的立即数域仅支持有符号模式，但偏移范围比 Arm64 大，而且是一个对称区间，为 -2048~2047，因此偏移的基准放置在栈帧哪一侧并不影响其寻址能力，新增实现的“例外”栈帧布局理论上完全可以替代原有“常规”布局，作为唯一的通用实现。

基于修复工作中实现的栈帧布局手动选择管理机制，强制所有被编译函数使用新布局，同样使用源码库内置功能性测试集进行全面测试，结果完全一致，未引入新的测试用例失败，初步表明该方案可行。不过由于该方案改动较大，且栈帧布局关联众多、影响范围大，产生意外问题的可能性也较大，还需进一步研究、验证。

进一步分析，此处栈帧布局设计还有再延伸优化的可能。ld/st 指令立即数偏移寻址范围的对称性，使得偏移基准放置在栈帧两侧其寻址能力相同的同时，也都是浪费了其一半的能力：帧指针 FP 放置在栈帧高地址一端时，访存读写帧内

数据用不到立即数的正值区间，FP 放置在低地址一端时则用不到其负值区间。而对于超出立即数支持范围的访存则需要通过算术运算另行单独构造地址，需要使用额外的寄存器和更多的指令辅助。因此，可考虑在维持 StackGuard 机制有效的前提下，将 FP/RA 寄存器组再调整保存至尽量更靠近栈帧中间但又在 GS Cookie 之上的位置，以更加充分地利用 ld/st 指令中的立即数域，更加高效地读写栈帧内容，提升性能。

3.5 本章小结

为应对经典的缓冲区溢出安全问题，编译器领域广泛使用 StackGuard 技术检测防御，.NET 底层的 RyuJIT 编译器也不例外。但是对于 localloc 特殊缓冲区溢出的场景，LoongArch64 上的 RyuJIT 编译器内置 StackGuard 机制没有如期发挥作用，造成安全漏洞。

设计验证程序进行实验，在 LoongArch64 和适配更为充分的 Arm64 架构平台上同一程序的运行结果不同。结合 RyuJIT 编译器内部实现，分析判断是 LoongArch64 平台上的栈帧布局设计存在缺陷，根源在于其中的帧指针 FP/返回地址 RA 寄存器组的排布位置不当。为应对该 localloc 特殊缓冲区场景，新增实现了一种例外情况栈帧布局，并同步调整了与之关联的函数序言/尾声指令生成逻辑。漏洞修复完成后使用例程和官方 runtime 功能性测试集进行测试验证，结果均正常符合预期。

此外，结合 LoongArch64 指令集中 load/store 指令的特性进一步分析，发现栈帧布局方案仍有后续调整以实现优化的空间，且初步验证具有可行性，但限于研发工作实际条件等暂未落实，可以作为未来的改进参考。

第 4 章 硬件加速特性支持

为了提高生成代码的性能，编译器除了可以在软件层面优化编译算法，还可以利用目标硬件架构的特性进行加速，如 SIMD（单指令流多数据流，Single Instruction Multiple Data）、多核并行、GPU 加速、专用硬件指令、功耗管理等。LoongArch 平台上 .NET 底层 RyuJIT 编译器先前实现的版本，尚未支持指令集的向量扩展部分，对一些基础指令集中面向特殊场景计算（如比特序列前导 0 的计数）的高效指令的利用也较为有限，无法充分发挥硬件的性能潜力。

本章阐述为 LoongArch 架构上的 .NET 添加硬件内部函数和向量 SIMD 操作特性的相关工作，其中 4.1 节简要介绍硬件内部函数与 SIMD，4.2 节阐述 .NET 中该特性的相关设计原理及添加支持需要做的作品内容，4.3 节给出在现有条件下为 LoongArch 架构添加相关支持的具体实现过程，4.4 节给出添加特性支持的结果，4.5 节对本章内容作简要总结。

4.1 硬件内部函数与 SIMD 简介

硬件内部函数（Hardware Intrinsic）是现代计算机体系结构和编译器领域中的一项重要技术，它允许高级编程语言直接映射并利用特定处理器或硬件平台的底层特性，最大限度地发挥目标硬件的性能潜力，同时又使代码保持较高的可读性和可维护性，在高级抽象和低级汇编编程之间取得了较好的平衡。硬件内部函数通常表现为一系列经过封装的特殊函数或方法，以保持与编程语言其他特性的良好集成，这些函数或方法能够精确地映射到特定于硬件架构的指令级别操作，包括一些基本数学运算、二进制比特位处理，以及加速向量运算的 SIMD 操作等。

其中 SIMD 操作是硬件内部函数中数量最多、当今应用最为广泛的一类。单条 SIMD 向量指令一次性同时处理多个数据元素，提供显著的计算并行性，可使硬件在处理计算密集型任务时获得大幅加速。一个简单的 SIMD 运算如图 4.1 所示，两个三维向量相加，若使用标量算术加法指令的常规方式，需要每个维度分别计算，需三条指令计算 3 次（多数情况还需要额外的数据搬运操作），而 SIMD 操作使用向量加法指令配合专用寄存器，只需一条指令计算 1 次，所需处理器时钟周期大幅减少，极大提高数据处理时的吞吐量。

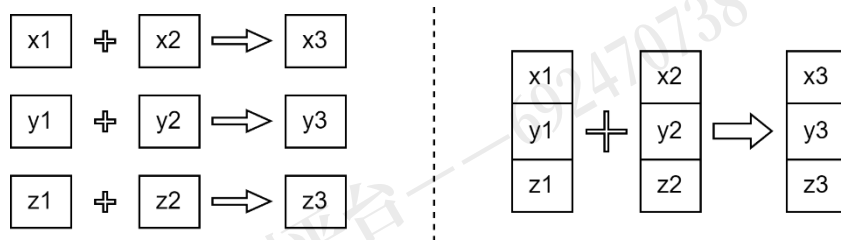


图 4.1 两个三维向量相加的常规标量实现（左）和 SIMD 实现（右）

硬件内部函数通过暴露接口，使得上层程序员可以通过高级编程语言访问原本需要通过汇编语言才能使用的底层硬件特性，减少处理器计算过程中大量不必要的数据搬运和转换开销，或是将一些算法逻辑从由多条基本的通用运算指令组合实现改为由单条特殊指令直接支持，从而极大地提高程序执行效率，实现性能的优化提升，在高性能计算、多媒体处理、加密算法、机器学习等应用领域具有显著优势。硬件内部函数的最终落地实现，仍依靠编译器完成从编程接口到机器指令的映射转换，但相比于另一种将特殊指令的映射也完全交由编译器实现（如自动向量化）的方式，通过硬件内部函数向开发者暴露接口使得编译器自身复杂度增加的更少、编译时间成本更低（更适合于 JIT 即时编译）、最终效果也更有保障。现代主流编译器如 GCC^[26]、LLVM^[27]等已普遍支持。

包含 SIMD 操作在内的硬件内部函数与硬件架构和指令集紧密相关，在不同的硬件平台往往有不同的设计和支持程度。现代处理器大多通过相应架构的指令集扩展（一组硬件指令的集合，通常也称为 Instruction Set Architecture, ISA）来提供对应指令，如 X86 架构的处理器有 SSE、AVX、BMI 等部分，Arm 架构的处理器有 AdvSimd、ARMv8.2-DotProd、Crc32 等部分，而 LoongArch 架构则有 LSX、LASX 等。各架构的 ISA 不仅有指令本身的编码定义等固有差异，所支持的硬件内部函数的种类、数量等也并非完全一致。目标硬件平台间的互不兼容导致编译器往往也需要分别进行针对性适配。

4.2 .NET 中的相关设计

在很长一段时间内，以 SIMD 操作为代表的硬件内部函数只实现在 C/C++ 等平台原生的语言生态中，其取得的成功为跨平台的 .NET 提供了范例和指导。作为托管运行时，.NET runtime 为生态内编程语言提供比 C/C++ 等更高级别的抽象，可以在发挥硬件内部函数性能优势的同时提供更友好的编程体验。

.NET 通过标准库中的 API 向上层应用提供访问硬件特性的功能始于 2014 年的 .NET Framework 4.5.2，首次公开一些通用的向量 API 以利用硬件的部分 SIMD 指令和寄存器^[28]。2019 年推出的 .NET Core 3.0 进一步扩展，将通用向量

API 无法利用的更多 SIMD 指令以及一些非向量类型的硬件平台相关的特殊指令也暴露给开发者，并将对这些指令的访问机制正式称为硬件内部函数 `hardware intrinsics`，还基于其设计实现了新的固定大小的通用向量类型及操作^[29]。在后续的版本迭代中，.NET 提供的硬件内部函数的 API 数量和支持的硬件 ISA 不断得以丰富^[30]。这些 API 可在上层编程语言（C#等）中调用，在底层由 RyuJIT 编译器直接识别转换为相应的硬件指令，避免产生函数调用开销，为现实应用程序如 ML.NET 机器学习框架带来了显著的性能提升^[31]。

.NET 中对硬件内部函数和向量 SIMD 操作两个特性的支持机制是紧密耦合的，两者共属于更广义的内在函数（`intrinsics`），其现在的关系如图 4.2 所示。

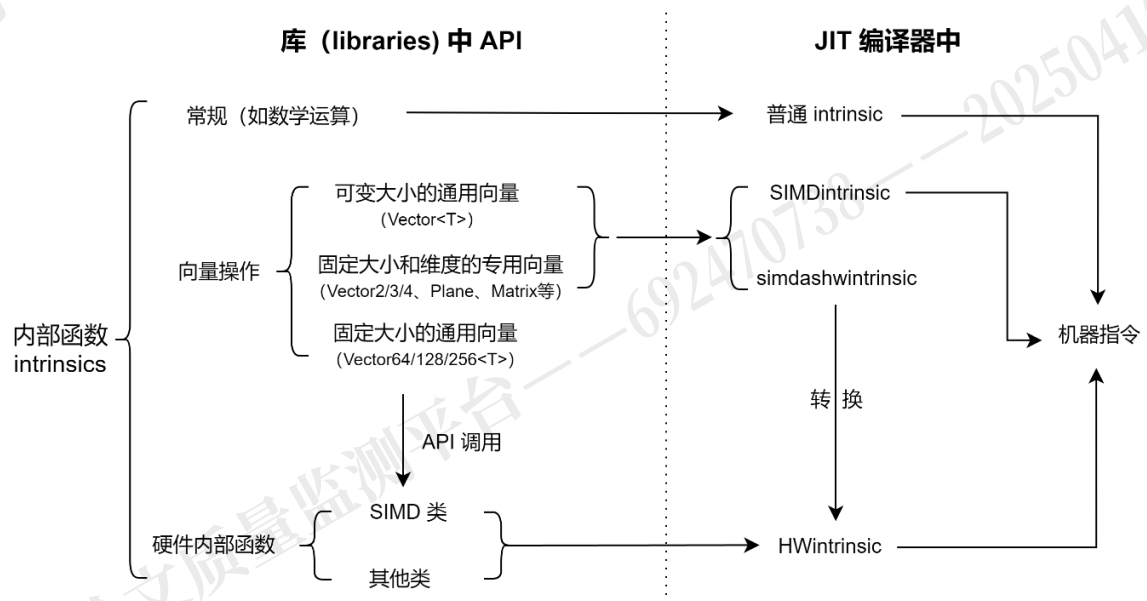


图 4.2 .NET 中硬件内部函数与向量 SIMD 操作的关系

硬件内部函数 API 在标准库中实现于 `System.Runtime.Intrinsics` 命名空间下对应于各指令集架构（x86、Arm、LoongArch）的嵌套子命名空间内，直接暴露底层硬件的大量指令接口（既包括 SIMD 指令也包括一些其他特殊指令），每个 API 一般与机器指令具有一对一的映射关系，与硬件平台强相关，且不提供软件回退，根据所属指令集 ISA 分类设计实现。

向量 SIMD 操作相关 API 在库函数中分散在两个命名空间下的多个 `struct` 结构体/class 类上，可分为两大部分：

- 1) `System.Numerics` 命名空间下，定义了 ①总大小与硬件平台相关（在运行时确定）、抽象程度更高、开发者用起来更简单方便但支持的操作种类相对有限的通用向量类型 `Vector<T>` 和 ②一些显式固定维度和基础数据类型（因而总大小也被固定）的常用具体向量类型（`Vector2`、`Vector3`、`Vector4`、`Matrix3x2`、`Matrix4x4`、`Plane`、`Quaternion` 等）。这些向量类型

的全部 API 都跨平台通用,即使其中的 `intrinsic` 类方法也仅在 RyuJIT 编译器内部才与硬件平台相关联,且对硬件特性的利用程度较为有限,仅用到部分基础的 SIMD 指令。

- 2) `System.Runtime.Intrinsics` 下,定义了总大小固定(64、128、256 位)的通用向量类型 `Vector64<T>`、`Vector128<T>`、`Vector256<T>`,其 API 方法除了创建一类的操作(`Create***`)之外直接暴露的功能非常少,主要用于配合硬件内部函数的实现。这些向量类型的全部 API 也都跨平台通用,其内部实现优先调用硬件内部函数 API 加速,并提供软件回退以供目标硬件不支持相应 ISA 时使用。

在 .NET 中支持硬件内部函数和向量 SIMD 操作这些硬件加速特性,除了在库中定义实现上述相关 API 外,还需在运行时后台添加对应支持,包括在虚拟机(执行引擎)中定义并识别目标平台相关配置信息,以及改造 RyuJIT 编译器的多个环节/模块以顺利将其转换映射为最终的机器指令^[32]。

- (1) 从虚拟机和环境配置中获取目标硬件平台的信息,以确定可以生成哪些指令。

- (2) 由于硬件内部函数和向量 SIMD 操作大量依赖于特定指令集 ISA 的专用指令,需要添加相应指令的编码定义。

- (3) 为各种抽象向量类型和平台支持的硬件内部函数维护一系列 `intrinsic` 表,记录支持的 `intrinsic` 清单和其与指令的映射关系等信息。

- (4) 定义相关的语法树 IR 结点类型, `GenTreeSIMD`、`GenTreeHWIntrinsic` 等。

- (5) 在编译前端的 IL 导入解析(`import`)阶段,综合 `intrinsic` 表和当前环境信息,从所有的 `method` 方法调用中识别出支持的 `intrinsic` 调用,并映射生成相应语法树 IR 结点。

- (6) 在编译后端的低级化(`lowering`)阶段,对相关语法树 IR 结点进行一些包含性检查和变换,暴露其寄存器需求。

- (7) 在编译后端的寄存器分配(`LSRA`)阶段,识别相关语法树 IR 结点中的寄存器引用并为之构造引用位置 `RefPosition`,进而结合 ABI 要求等因素合理分配向量寄存器。

- (8) 在编译后端的汇编指令生成(`CodeGen`)和机器码发射(`Emitter`)阶段,将相关语法树 IR 结点映射为最终指令并发出。

其中有一个特殊设计:由于每个硬件内部函数通常只映射为单条硬件指令(或一个短序列),其可以更简单地经历各个编译器阶段,而不需要一般语法树

节点的复杂变换逻辑，因此在编译器中其他功能或代码只要可行也应尽量利用硬件内部函数的处理机制，向量 SIMD 操作便是典型，于是 RyuJIT 编译器中将一大部分 SIMD 操作节点尽早地解析转换为了硬件内部函数节点（SIMD_AS_HWINTRINSIC）。

.NET 中这些内容的整体设计框架是各指令集架构共用的，但仍有大量的具体实现细节与硬件平台相关，仍需要针对性适配。在本文工作开展时所基于的基线 .NET 6.0 版本中，x86/x64 和 Arm/Arm64 架构已经获得较为充分的支持，而 LoongArch 则还完全未添加相关实现，本文工作旨在填补这一空缺。

4.3 为 LoongArch 添加特性支持的实现

如前文所述，.NET 中硬件内部函数和向量 SIMD 操作硬件加速特性的支持主要涉及两大部分内容——在运行时库中添加 API 的定义实现、改造 RyuJIT 编译器实现运行时后台支持，后者为工作重点。

4.3.1 库函数接口

为 LoongArch 添加硬件内部函数 API，需要从无到有创建特定于平台架构的子命名空间 `System.Runtime.Intrinsics.LoongArch`，在其中根据指令集架构的组成划分定义不同的类（class）表示各 ISA，包括 `LoongArchBase`、`LSX`、`LASX` 等。每个类中都公开一个 `IsSupported` 静态属性以指示运行时所在的硬件平台是否支持该 ISA，以及一系列映射到该 ISA 所含指令的方法（检测到硬件不支持时抛出异常）。各指令集架构中包含的指令功能不同，导致可暴露提供的接口也并不一致，在适配初期先以支持充分的 x86/Arm 架构实现情况为参考，优先添加它们已提供且 LoongArch 也有对应指令的接口，LoongArch 特有的指令相应接口留待后期优化阶段再加以补充完善。

向量 SIMD 操作相关 API 中，`System.Numerics` 命名空间下大小可变的通用向量（`Vector<T>`）和显式固定维度的常用具体向量（`Vector2/3/4`、`Matrix`、`Plane` 等）在库函数层面与硬件平台无关，其 API 不需要调整，保持已有设计即可；而 `System.Runtime.Intrinsics` 命名空间下针对固定大小的通用向量部分（`Vector64/128/256<T>`），在相关类（class）/结构体（struct）内部方法实现中优先调用硬件内部函数 API 加速的逻辑内，需添加对 LoongArch 对应 ISA 的检测和 API 的调用。此外 `System.Numerics` 命名空间下封装比特相关操作的工具类 `BitOperations`，内部方法实现也会类似地优先利用硬件内部函数 API 和固定大小通用向量的 SIMD 操作 API 来加速，也需为其添加相应于 LoongArch 的处理。

4.3.2 编译基础设施

项目源码自身的编译配置为 LoongArch 架构添加上向量 SIMD 操作和硬件内部函数特性相应的 `FEATURE_SIMD`、`FEATURE_HW_INTRINSICS` 等宏定义，将相关逻辑的代码纳入到最终的运行时、编译器产品中。

虚拟机（执行引擎）组件中添加了对硬件配置（如 CPU 支持哪些架构特性）和 ISA（如 LSX、LASX）的定义与识别逻辑，记录为运行期间的环境配置信息，并通过通信交互逻辑将其传递给 JIT 编译器，指导编译方向。

RyuJIT 编译器内部：

- （1）适配 SIMD 相关 ABI：定义了向量类型（`TYP_SIMD8`、`TYP_SIMD12`、`TYP_SIMD16`、`TYP_SIMD32`）的地址对齐规则、构成属性、函数调用过程中传参方式；定义了向量寄存器的位宽等数据、函数调用过程中的保存规则等。
- （2）定义 SIMD 类指令：在 `instrsloongarch64.h` 文件中新增了 LoongArch 指令集的 LSX 和 LASX 向量扩展部分共 1442 条指令的汇编码和机器码，并相应调整了内部描述指令的关键数据结构 `struct instrDesc`。
- （3）定义 intrinsics：在 `simdintrinsiclist.h` 文件中定义了 25 个 SIMD intrinsics，在 `hwintrinsiclistloongarch64.h` 文件中定义了 231 个 hardware intrinsics，在 `simdashwintrinsiclistloongarch64.h` 文件中定义了 82 个 SIMD as hardware intrinsics，这些清单和库中的 API 相对应，通过识别它们来选择带有硬件加速的编译路径；定义了 `struct HWIntrinsic`、`struct HWIntrinsicInfo` 等内部描述 intrinsic 的数据结构。

4.3.3 编译过程

编译过程各阶段（详见 1.2.2 小节）的处理逻辑尽量复用跨平台架构的已有公共代码，仅通过 `#if`、`#ifdef` 等宏定义改造、添加适配 LoongArch 平台相关部分，涉及关键处理步骤如下：

（1）前端导入（import）阶段

由于不同架构的指令本身固有属性（如立即数范围、运算规则）、支持的 intrinsic 清单等存在区别，从字节码导入阶段开始对于一些硬件内部函数的处理细节就不完全一致，此阶段改造或添加的编译函数主要有 `Compiler::impHWIntrinsic()`、`Compiler::impSpecialIntrinsic()`、`Compiler::impSimdAsHWIntrinsicSpecial()`、`Compiler::impSimdAsHWIntrinsicCndSel()`。

(2) 后端低级化 (lowering) 阶段

此阶段与平台架构的关联性又开始增大,添加或改造的相关编译函数主要是低级化 SIMD 和 HWIntrinsic 语法树节点的逻辑: Lowering::LowerSIMD()、Lowering::ContainCheckIndir()、Lowering::ContainCheckSIMD()、Lowering::LowerHWIntrinsic()、Lowering::LowerHWIntrinsicCreate()、Lowering::LowerHWIntrinsicDot()、Lowering::LowerHWIntrinsicCmpOp()、Lowering::ContainCheckHWIntrinsic()。

(3) 后端寄存器分配 (LSRA) 阶段

添加或改造的相关编译函数主要是为 SIMD 和 HWIntrinsic 语法树节点构建引用位置 RefPositions 的逻辑 LinearScan::BuildSIMD()、LinearScan::BuildHWIntrinsic()。

(4) 后端目标代码生成 (CodeGen) 阶段

虽然从顶层设计来看,硬件内部函数 hardware intrinsic 对应的代码生成相当简单,只需将其一一映射为特定的硬件指令,但由于指令是完全平台相关的,且数量巨大,该部分的实际工作量却是最大的。此外,那些暂不适合转换为 hardware intrinsic 的 SIMD intrinsics 也需另额外行处理。因此,本阶段是适配支持的重点内容。添加或改造的相关编译函数主要有:

- ① 为未转换为 hardware intrinsic 的 SIMD 节点生成指令的 CodeGen::genSIMDIntrinsic()、CodeGen::getOpForSIMDIntrinsic()、CodeGen::genSIMDIntrinsicInit()、CodeGen::genSIMDIntrinsicInitN()、CodeGen::genSIMDIntrinsicUnOp()、CodeGen::genSIMDIntrinsicBinOp()、CodeGen::genSIMDIntrinsicNarrow()、CodeGen::genSIMDIntrinsicWiden()、CodeGen::genSIMDIntrinsicUpperSave()、CodeGen::genSIMDIntrinsicUpperRestore()、CodeGen::genMultiRegStoreToSIMDLocal() 等。其中 CodeGen::genSIMDIntrinsic() 是主入口,在内部根据节点在 SIMD intrinsic 清单表中的 Intrinsic Id 选择调用不同的实现。
- ② 为 HWIntrinsic 节点生成指令的 CodeGen::genHWIntrinsic(), hardware intrinsic 清单表中定义的每一项都在此处理(很多项的处理逻辑相似,分类实现尽量复用代码)。
- ③ 为一些其他涉及 SIMD 类型变量的节点生成指令的 CodeGen::genCodeForLclVar()、CodeGen::genCodeForStoreLclVar()、CodeGen::genCodeForLclFld()、CodeGen::genLoadIndTypeSIMD12()、CodeGen::genStoreIndTypeSIMD12()、CodeGen::genLoadLclTypeSIMD12()、CodeGen::genStoreLclTypeSIMD12()、CodeGen::genPutArgStk()、CodeGen::genMultiRegStoreToLocal()、CodeGenInterface::spillReg()、CodeGen::genUnspillRegIfNeeded()、

`CodeGen::genUnspillLocal()`、`CodeGen::genSIMDSplitReturn()` 等。

(5) 后端指令发射 (Emitter) 阶段

主要将目标代码生成阶段新选用的各种 SIMD 类汇编指令构造为最终的机器码并发射，以及在反汇编接口中增加对这些指令的支持。改造的相关编译函数主要有 `emitter::emitIns_S_R()`、`emitter::emitIns_R_S()`、`emitter::emitIns_R_I()`、`emitter::emitIns_Mov()`、`emitter::emitIns_R_R()`、`emitter::emitIns_R_R_I()`、`emitter::emitIns_R_R_R()`、`emitter::emitIns_R_R_I_I()`、`emitter::emitIns_R_R_R_R()`、`emitter::emitIns_S_R_SIMD12()`、`emitter::emitDisInsName()` 等。

(6) 其他阶段

包括一些 ABI 适配、优化等逻辑，较为零散，在此不再详述。

4.4 添加特性支持的结果

4.4.1 功能测试

经过一系列开发和调试工作，该特性支持已基本实现，runtime 源码整体可正常编译构建完成。

功能性系统测试采用前文 2.1 小节所述 runtime 源码库内置的测试集，构建生成后每个测试用例均对应一个 shell 脚本，指定构建好的 runtime 目录路径参数后，通过终端直接运行相应脚本即可开启测试，脚本运行后的返回值即为测试用例的测试结果。

测试结果显示，添加 SIMD 和硬件内部函数特性支持后的 runtime 的功能正确性维持了与支持该特性之前相同的水平，没有引入新的用例失败，除原有的极少数已知问题外的共 10044 个测试用例均测试通过，其中专门针对 SIMD 特性进行测试的用例有 113 个，位于 `tests/JIT/SIMD/` 目录下，针对硬件内部函数特性进行测试的用例有 358 个，位于 `tests/JIT/HardwareIntrinsics/` 目录下，分别摘选部分用例列举如表 4.1 所示。

表 4.1 针对 SIMD 和硬件内部函数的部分测试用例

类型	测试用例名
SIMD	BitwiseOperations_r/BitwiseOperations_r.sh
	CtorFromArray_ro/CtorFromArray_ro.sh Matrix4x4_r/Matrix4x4_r.sh
	MinMax_r/MinMax_r.sh
	Vector3_ro/Vector3_ro.sh
	Vector4_ro/Vector4_ro.sh
	VectorArrayInit_r/VectorArrayInit_r.sh
	VectorDot_r/VectorDot_r.sh
	VectorInit_r/VectorInit_r.sh
	VectorSqrt_ro/VectorSqrt_ro.sh
Hardware Intrinsics	General/NotSupported/NotSupported_r/NotSupported_r.sh
	General/Vector128_1/Vector128_1_ro/Vector128_1_ro.sh
	General/Vector256/Vector256_r/Vector256_r.sh
	General/Vector64/Vector64_ro/Vector64_ro.sh
	X86/Avx/Avx_r/Avx_r.sh
	X86/General/VectorArgs_r/VectorArgs_r.sh
	X86/Popcnt.X64/Popcnt.X64_r/Popcnt.X64_r.sh
	Arm/AdvSimd/AdvSimd_Part10_ro/AdvSimd_Part10_ro.sh
	Arm/ArmBase/ArmBase_r/ArmBase_r.sh Arm/Crc32/Crc32_r/Crc32_r.sh

4.4.2 性能测试

性能测试利用 .NET 社区的 performance 源码库^[33], 其中的 microBenchmark 部分用于测试运行时 runtime 的性能, 该测试基于 BenchmarkDotNet^[34]项目开发, 采用了项目隔离、进程隔离、调用次数自动调优、测试前预热、多次迭代、精度控制、自身防卫等机制以尽可能保证结果的准确性, 并输出丰富的统计数据, 在测试完成后还提供了结果数据比较工具, 用以比较不同版本运行时的性能。

测试步骤如下:

- 1) 预备工作: 克隆 performance 仓库源码, 切换为与 runtime 匹配的 release/6.0 分支, 并稍作修改 (主要添加龙芯 nuget 源、配置内部 charp 项目的目标框架为 net6.0); 安装依赖项 Python、dotnet SDK、libgdiplus; 准备好待测的构建好的 runtime。
- 2) 在源码的 benchmarks/micro/目录下运行 dotnet run 命令, 指定目标框架、测试项目、构建好的 runtime 路径等参数, 即可开始测试, 获得各测试项的运行时间相关数据。

- 3) 针对多个 runtime 版本测试完毕后,使用源码库内的 ResultsComparer 工具可对各独立 microbenchmark 测试的结果进行两两对比。

测试开展基于同一台采用龙芯 3A5000 处理器的台式计算机,内存为 16GB,操作系统为 Linux 4.19.0-18-loongson-3,每次测试均重启机器、不额外开启其他软件程序。

龙芯 3A5000 处理器主要技术参数如下:

- 核心数量: 4 个,单核单线程;
- 主频: 2.3GHz–2.5GHz;
- 核心特性: 支持 LoongArch64 指令集(含 128 和 256 位向量指令扩展),四发射乱序执行,具有 4 个定点单元、2 个向量单元和 2 个访存单元;
- 高速缓存 cache: 每个处理器核包含 64KB 私有一级指令缓存、64KB 私有一级数据缓存、256KB 私有二级缓存;所有处理器核共享 16MB 三级缓存。

分别测试添加 SIMD 和硬件内部函数特性支持前、后的 runtime,在 2% 的常用统计检验阈值下对比结果数据,记添加特性之前的为 base、添加特性之后的为 diff, diff 运行更快(时间更短、性能更好)的测试项有 825 个,运行时间多次迭代的中位数的比值(base/diff)的几何平均 geomean 为 1.708, diff 运行更慢的测试项有 921 个,运行时间多次迭代的中位数的比值(diff/base)的几何平均 geomean 为 1.120。将两部分数据汇总整合,计算所有差异测试项全体的运行时间多次迭代的中位数的比值(base/diff)的几何平均 geomean 如下,结果约为 1.213。

$$\left(1.708^{825} \times \left(\frac{1}{1.120}\right)^{921}\right)^{\frac{1}{825+921}} \approx 1.213$$

这表明,对于 microBenchmark 测试集,添加该特性支持后 runtime 的综合性能提升到了之前的约 1.213 倍。

上述差异测试项中,运行时间降低幅度最大、性能提升最明显的前 30 个的测试数据如表 4.2 所示,可见大部分与向量相关,其中,即使以较为谨慎的态度质疑测量值精度误差,忽略掉小于 1 ns 的数值,其中性能提升最大的测试项 System.Numerics.Tests.Perf_VectorConvert.Narrow_ushorty 的运行时间降低幅度也达到了约 50 倍的显著级别。

表 4.2 添加硬件加速特性支持后性能提升最大的 microBenchmark 测试项

测试项	Base / diff	Base Median (ns)	Diff Median (ns)
System.Memory.Constructors<Byte>.SpanFromArrayStartLength	Infinity	0.62	0.00
System.Memory.Constructors<Byte>.ReadOnlySpanFromArrayStartLength	Infinity	0.40	0.00
System.Memory.MemoryMarshal<Byte>.GetReference	Infinity	0.60	0.00
System.Memory.MemoryMarshal<Byte>.CastToByte	Infinity	0.43	0.00
System.Memory.MemoryMarshal<Int32>.CastToByte	Infinity	0.82	0.00
System.Numerics.Tests.Constructor.ConstructorBenchmark_SByte	Infinity	1.03	0.00
System.Numerics.Tests.Constructor.ConstructorBenchmark_Int16	Infinity	1.04	0.00
System.Numerics.Tests.Constructor.ConstructorBenchmark_UInt32	Infinity	1.03	0.00
System.Numerics.Tests.Perf_Quaternion.LengthBenchmark	Infinity	0.37	0.00
System.Numerics.Tests.Perf_Vector2.CreateFromScalarXYBenchmark	Infinity	2.80	0.00
System.Numerics.Tests.Perf_Vector2.OneBenchmark	Infinity	10.40	0.00
System.Numerics.Tests.Perf_Vector2.UnitYBenchmark	Infinity	10.40	0.00
System.Numerics.Tests.Perf_Vector3.CreateFromScalarXYZBenchmark	Infinity	5.60	0.00
System.Numerics.Tests.Perf_Vector4.CreateFromScalarXYZWBenchmark	Infinity	5.61	0.00
System.Numerics.Tests.Perf_Vector2.UnitXBenchmark	2285.23	10.42	0.00
System.Numerics.Tests.Constructor.ConstructorBenchmark_Int64	1391.69	0.82	0.00
System.Numerics.Tests.Constructor.ConstructorBenchmark_UInt16	686.82	0.84	0.00
System.Numerics.Tests.Constructor.ConstructorBenchmark_Single	570.79	1.03	0.00
System.Numerics.Tests.Perf_Vector2.CreateFromScalar	553.89	2.80	0.01
System.Numerics.Tests.Perf_VectorConvert.Narrow_ushort	49.58	40547.55	817.87
System.Numerics.Tests.Perf_VectorConvert.Narrow_short	49.53	40537.63	818.47
System.Numerics.Tests.Perf_VectorConvert.Widen_sbyte	41.41	42221.16	1019.69
System.Numerics.Tests.Perf_VectorConvert.Widen_byte	41.35	42115.69	1018.53
System.Numerics.Tests.Perf_Vector2.AbsBenchmark	40.37	16.19	0.40
System.Numerics.Tests.Perf_VectorConvert.Narrow_int	36.34	29727.32	818.07
System.Numerics.Tests.Perf_VectorConvert.Narrow_uint	36.30	29707.86	818.44

(续表)

测试项	Base / diff	Base Median (ns)	Diff Median (ns)
System.Numerics.Tests.Perf_VectorConvert.Convert_float_int	34.94	25068.69	717.46
System.Numerics.Tests.Perf_VectorConvert.Convert_uint_float	34.36	28070.59	816.87
System.Numerics.Tests.Perf_VectorConvert.Widen_short	32.00	32573.55	1017.84
System.Numerics.Tests.Perf_VectorConvert.Narrow_ulong	31.94	26198.07	820.31

注：base 和 diff 分别代指对添加硬件加速特性前、后的 runtime 进行的测试。

4.4.3 后续工作方向

硬件内部函数和向量 SIMD 操作特性目前仅是实现初步支持，许多地方实现得较为粗糙，还需开展系统性分析优化。

标准库 API（及其实现）清单主要是对标着 x86 和 Arm 架构的清单，以尽量向其看齐、提供与成熟架构一致的体验为目标而添加的，未结合 LoongArch 架构自身特点进行针对性精细设计，可进一步完善，去除掉部分效果不好的（若有），以及添加一些 LoongArch 特有的。

功能性测试集中针对硬件内部函数的相当一部分测试用例是针对各平台架构单独设计的（参见表 4.1），而 LoongArch 还未添加，因此实际上测试的覆盖性还有所欠缺，需加以补充，更充分地开展测试验证。

性能测试集 Performance 及其底层依赖组件在 LoongArch 上的适配还不完善，测试的稳定性、准确性和全面性有待分析和加强。

4.5 本章小结

硬件内部函数和向量 SIMD 操作是两个重要的硬件加速特性，在业界有广泛应用，.NET 近年也添加了相关支持，但由于该特性与硬件平台架构强相关，需要分别适配，而 LoongArch 是一个新生架构，尚未被纳入其中。

调研分析 .NET 中支持该特性的整体设计原理后，开展了适配工作，包括在标准库中添加 API 定义和改造底层 JIT 编译器。虽然整体框架各架构可共用，但仍有大量实现细节不得不针对 LoongArch 单独实现，最终添加了 1442 条向量扩展指令，新增支持了 231 个硬件内部函数，加速了 82 个中途转换为硬件内部函数的向量 SIMD 操作和 25 个常规向量 SIMD 操作，覆盖面与 x86、Arm 架构达

到了同等水平。

开发完成后进行测试，结果显示运行时 `runtime` 的功能正确性维持了与支持该特性之前相同的水平，没有引入新的用例测试失败；而性能方面，基于 `microBenchmark` 测试集，添加该特性支持后 `runtime` 的综合性能提升到了之前的约 1.213 倍，其中单个测试项性能提升的最大幅度更是达到了约 50 倍。但同时也不可否认，当前实现的支持仍是比较初步的，还有诸多不足，后续需要继续优化完善。

第 5 章 总结与展望

5.1 本文工作总结

LoongArch 架构上的 .NET 此前已基本移植完成，但适配得还不够充分，在性能、功能特性完整度上都还有较大优化空间。本文主要关注其 CoreCLR 运行时中的底层 JIT 编译器（RyuJIT）模块，在先前版本（6.0 版本）的基础上结合 LoongArch 架构特性开展了一些相关的分析与完善、优化工作：

（1）分析 JIT 编译生成指令的质量：

从批量统计应用程序编译所得静态指令的数量入手，对比 LoongArch 和 Arm 上结果数据的差异，发现 LoongArch 上确实存在较大劣势，相同程序编译生成的指令数量普遍更多。进一步，详细分析了一些典型用例程序的指令序列，对访存寻址方式、大块连续内存初始化、小尺寸数据类型在寄存器中的表示、转移指令的利用等场景或问题展开探究，提出了一些较为具体的优化构想，供未来继续改进完善 RyuJIT 编译器参考。

（2）发现、分析、修复了一个缓冲区溢出防护机制失效的安全漏洞：

发现该漏洞后首先通过设计实验，验证了其存在。分析研究该漏洞的原理，发现根源在于栈帧布局设计存在缺陷，帧指针 FP/返回地址 RA 寄存器组的排布位置不当，导致在 `localloc` 特殊缓冲区溢出的场景下，编译器内置的 `StackGuard` 机制没有正常起效，在函数返回地址被溢出篡改时无法及时检测到并开启防护。为修复该漏洞，调整了栈帧布局的设计，新增实现了一种例外情况布局作为应对。修复完成后进行了测试验证，结果正常符合预期。此外，结合 LoongArch 指令特性，拓展延伸提出了进一步优化栈帧布局的方向。

（3）添加支持了硬件内部函数和向量 SIMD 操作硬件加速特性：

调研分析了该特性的整体设计原理后，通过在标准库中添加 API 定义和改造编译器，实现了初步支持。共添加了 1442 条向量扩展指令，新增支持了 231 个硬件内部函数，加速了 82 个中途转换为硬件内部函数的向量 SIMD 操作和 25 个常规向量 SIMD 操作，覆盖面与 x86、Arm 等成熟架构达到了同等水平。测试结果显示，实现该特性支持后运行时 `runtime` 的功能正确性维持了与实现支持前相同的水平，而综合性能则提升到了之前的约 1.213 倍，其中单个测试项性能提升的最大幅度更是达到了约 50 倍。

由于暂无其他团队从事相关工作，本文对于该工程项目所做的上述三方面工

作及成果在一定程度上是“开创性的”，其中指令分析部分的成果为未来继续改进完善 RyuJIT 编译器提供参考，具有一定的指导意义；漏洞修复和硬件加速特性支持的成果已落实于代码，进入 LoongArch 平台的 .NET 运行时产品内部，提升了其安全性和性能，已经或即将发布、投入实际应用。

5.2 未来展望

如前所述，本文对 LoongArch 上的 .NET 底层 JIT 编译器开展了一些分析与完善、优化工作，并取得了一定的积极成果。但具体所做的三方面工作，仍有推进深入、优化、完善的空间，未来还需要继续迭代、查漏补缺，让 LoongArch 平台上的 .NET 更加成熟、表现更加优秀。具体需要做的内容有：

- JIT 编译生成指令分析与优化：适配用于统计动态指令等的各种生态辅助工具，助力后续研发；分析更多场景的指令序列、探究更多优化方向，并将优化方案落地实现。
- 缓冲区溢出安全漏洞的修复：结合架构特性，实现对栈帧布局设计的进一步优化，提升性能。
- 硬件加速特性的支持：结合 LoongArch 特点，精细优化该特性的编译部分实现细节和 API 清单设计；补充添加功能性测试用例，提高测试覆盖率；完善性能测试工具的适配，让测试结果更可靠。

LoongArch 和 .NET 都仍在不断发展，如添加新的指令、新的功能特性或优化一些功能的实现机制等，在 LoongArch 上适配支持 .NET 的工作也不会一劳永逸，需要持续进行改进，相信随着研发工作的不断深入，LoongArch 平台上的 .NET 表现一定会追平乃至赶超其他成熟架构，助力国产自主软件生态的繁荣。

参考文献

- [1] 龙芯中科公司官方网站[EB/OL]. [2025-03-30]. <https://loongson.cn/>.
- [2] 胡伟武, 汪文祥, 吴瑞阳, 等. 龙芯指令系统架构技术[J]. 计算机研究与发展, 2023, 60(1): 2-16.
- [3] 龙芯架构 (LoongArch) 参考手册下载地址[EB/OL]. [2025-03-31]. <https://loongson.cn/download/index>.
- [4] 龙芯架构 ELF psABI 规范[EB/OL]. [2024-01-27]. <https://loongson.github.io/LoongArch-Documentation/LoongArch-ELF-ABI-CN.html>.
- [5] Microsoft. .NET 官方网站[EB/OL]. [2025-03-29]. <https://dotnet.microsoft.com/zh-cn/>.
- [6] ECMA-335 - ecma international[EB/OL]. [2025-03-30]. <https://ecma-international.org/publications-and-standards/standards/ecma-335/>.
- [7] ISO/IEC 23271:2012[EB/OL]. [2025-03-30]. <https://www.iso.org/standard/58046.html>.
- [8] Mono Project[EB/OL]. [2025-03-30]. <https://www.mono-project.com/>.
- [9] .NET runtime 源码库[CP/OL]. .NET Platform, 2025[2025-04-05]. <https://github.com/dotnet/runtime>.
- [10] 井田. CoreCLR 虚拟机在 MIPS 平台的移植与优化[D/OL]. 北京: 中国科学院大学, 2020[2024-01-27]. <https://d.wanfangdata.com.cn/thesis/ChJUaGVzaXNOZXdTmJAYnNDAxMDkSCFkzODUxMDI5GghrNDF0MXZqeA%3D%3D>.
- [11] 龙芯上的.NET (LoongArch 架构 .NET SDK 官方发布网站) [EB/OL]. [2025-03-31]. <https://www.loongnix.cn/zh/api/dotnet/>.
- [12] Aleph One. Smashing The Stack For Fun And Profit[J]. Phrack, 1996, 7(49).
- [13] Free Software Foundation, Inc. Automatic Storage with Variable Size[EB/OL]. [2024-03-15]. https://www.gnu.org/software/libc/manual/html_node/Variable-Size-Automatic.html.
- [14] Microsoft. OpCodes.Localloc Field[EB/OL]. [2024-03-15]. <https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.opcodes.localloc?view=net-6.0>.
- [15] Microsoft. stackalloc expression[EB/OL]. (2023-11-17)[2024-03-21]. <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/stackalloc>.
- [16] PaX Team. Address space layout randomization[EB/OL]. (2003-03-15)[2024-03-21]. <https://pax.grsecurity.net/docs/aslr.txt>.
- [17] Free Software Foundation, Inc. Program Instrumentation Options[EB/OL]. [2024-03-21]. <https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html#index-fstack-protector>.

- [18] WAGLE P, COWAN C. StackGuard: Simple Stack Smash Protection for GCC[J]. 2003.
- [19] The LLVM Project. Clang command line argument reference[EB/OL]. [2024-03-21].
<https://clang.llvm.org/docs/ClangCommandLineReference.html#cmdoption-clang-fstack-protector>.
- [20] Microsoft. /GS (Buffer Security Check)[EB/OL]. (2021-08-03)[2024-03-21].
<https://learn.microsoft.com/en-us/cpp/build/reference/gs-buffer-security-check?view=msvc-170>.
- [21] BIERBAUMER B, KIRSCH J, KITTEL T, et al. Smashing the Stack Protector for Fun and Profit[C/OL]//JANCZEWSKI L J, KUTYŁOWSKI M. 33th IFIP International Conference on ICT Systems Security and Privacy Protection (SEC): Vol. 529. Poznan, Poland: Springer International Publishing, 2018: 293-306[2024-03-19]. <https://inria.hal.science/hal-02023742>. DOI:10.1007/978-3-319-99828-2_21.
- [22] COWAN C, PU C, MAIER D, et al. StackGuard: Automatic adaptive detection and prevention of buffer-overflow attacks[C]//7th USENIX Security Symposium, January 26, 1998 - January 29, 1998. San Antonio, TX, United states: USENIX Association, 1998.
- [23] SZEKERES L, PAYER M, Tao Wei, et al. SoK: Eternal War in Memory[C/OL]//2013 IEEE Symposium on Security and Privacy. 2013: 48-62[2024-03-22].
<https://ieeexplore.ieee.org/document/6547101>. DOI:10.1109/SP.2013.13.
- [24] .NET Foundation. Arm64 JIT 栈帧布局设计文档[EB/OL]. (2020-03-10)[2024-03-28].
<https://github.com/dotnet/runtime/blob/release/6.0/docs/design/coreclr/jit/arm64-jit-frame-layout.md>.
- [25] Arm Limited. Arm Architecture Reference Manual for A-profile architecture[EB/OL]. (2023-04-21)[2024-03-28]. <https://developer.arm.com/documentation/ddi0487/ja/?lang=en>.
- [26] Free Software Foundation, Inc. Built-in Functions Specific to Particular Target Machines[EB/OL]. [2024-04-05]. <https://gcc.gnu.org/onlinedocs/gcc-13.2.0/gcc/Target-Builtins.html>.
- [27] The LLVM Project. LLVM Language Reference Manual[EB/OL]. [2024-04-05].
<https://llvm.org/docs/LangRef.html#intrinsic-functions>.
- [28] Immo Landwerth. The JIT finally proposed. JIT and SIMD are getting married[EB/OL]. (2014-04-07)[2024-04-06]. <https://devblogs.microsoft.com/dotnet/the-jit-finally-proposed-jit-and-simd-are-getting-married/>.
- [29] Tanner Gooding. Hardware Intrinsics in .NET Core[EB/OL]. (2019-09-03)[2024-04-06].
<https://devblogs.microsoft.com/dotnet/hardware-intrinsics-in-net-core>.
- [30] Tanner Gooding. Hardware Intrinsics in .NET 8[EB/OL]. (2023-12-11)[2024-04-06].

- <https://devblogs.microsoft.com/dotnet/dotnet-8-hardware-intrinsics/>.
- [31] Brian Lui. Using .NET Hardware Intrinsics API to accelerate machine learning scenarios[EB/OL]. (2018-10-10)[2024-04-08]. <https://devblogs.microsoft.com/dotnet/using-net-hardware-intrinsics-api-to-accelerate-machine-learning-scenarios/>.
- [32] EIDT C, GOODING T. SIMD support in .NET: abstract and concrete vector types and operations[C/OL]//Proceedings of the 18th ACM/IEEE International Symposium on Code Generation and Optimization. New York, NY, USA: Association for Computing Machinery, 2020: 229-241[2024-01-27]. <https://dl.acm.org/doi/10.1145/3368826.3377926>. DOI:10.1145/3368826.3377926.
- [33] .NET performance 源码库[CP/OL]. .NET Platform, 2025[2025-04-05]. <https://github.com/dotnet/performance>.
- [34] BenchmarkDotNet 官方网站[EB/OL]. [2025-04-01]. <https://benchmarkdotnet.org/>.